VBACKUP Saveset Utility for Linux

User’s Guide

VBACKUP-UG-0125

VBACKUP Version X01-25 *** 27-OCT-2026

Copyright © 2026 StarLet Squad and Ruslan R. Laishev.

VBACKUP is licensed under the Apache License, Version 2.0 (the ”License”); you may not use it except in compliance with the License. A copy of the License is in the file LICENSE of the kit, and at http://www.apache.org/licenses/LICENSE-2.0.

Unless required by applicable law or agreed to in writing, VBACKUP is distributed under the License on an ”AS IS” BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

Contents

Preface **4**
Intended Audience . . . 4
Why VBACKUP Exists . . . 5
Document Structure . . . 6
Related Documents . . . 7
Acknowledgments . . . 8
License . . . 9
Trademarks . . . 10
Conventions . . . 11

Chapter 1. Introduction **12**
1.1 What VBACKUP Does . . . 12
1.2 What a Saveset Gives . . . 13
1.3 What VBACKUP Does Not Do . . . 14
1.4 The Programs of the Kit . . . 15

Chapter 2. Installation **16**
2.1 Requirements . . . 16
2.2 Building and Installing . . . 17
2.3 Installing on Windows . . . 18
2.4 Installing in Another Place . . . 19
2.5 Verifying the Installation . . . 20
2.6 Removal . . . 21

Chapter 3. Concepts **22**
3.1 Savesets . . . 23
3.2 Operations . . . 24
3.3 Saveset Structure . . . 25
3.4 File Specifications and Wildcards . . . 27
3.5 Incremental Backup and the Journal . . . 28
3.6 Physical and Image Operations . . . 29
3.7 Compression . . . 30
3.8 Encryption . . . 32
3.9 Pipes . . . 34
3.10 Savesets on Other Nodes . . . 35
3.11 The Page Cache . . . 37
3.12 Threads . . . 38
3.13 The Stand-Alone Extractor and the Extractors of Last Resort . . . 39
3.14 File Manager Plugins . . . 40
3.15 Scheduling and Rotation . . . 41
3.16 OpenVMS BACKUP Savesets . . . 42
3.17 VBACKUP on Windows . . . 43

Chapter 4. Examples **45**

Chapter 5. Command Dictionary **49**

5.1 The VBACKUP Command 50
5.2 Qualifiers 54

Chapter 6. Messages **74**
6.1 Message Format 74
6.2 Severities and the Exit Status 75
6.3 Messages 76

Chapter 7. Troubleshooting **93**
7.1 The Verify Fails on Files That Are in Use 94
7.2 Owners and Attributes Are Not Restored 95
7.3 A Saveset Is Damaged 96
7.4 A Volume Is Missing or Belongs to Another Saveset 97
7.5 The Passphrase Is Asked for, Refused or Missing 98
7.6 A Saveset on Another Node Cannot Be Reached 99
7.7 An Incremental Restore Is Refused 100
7.8 A Device Is Refused 101
7.9 HELP Shows Only a Summary 102
7.10 A File Manager Does Not Open a Saveset 103

Appendix A. Files **104**
A.1 Files Installed 104
A.2 Files Used 105

Appendix B. Saveset Format Summary **106**
B.1 Conventions 107
B.2 Volumes and Blocks 108
B.3 Block Header (64 bytes) 109
B.4 Block Types 110
B.5 Records 111
B.6 Tags 112
B.7 Encryption (format.md, 6.10) 113
B.8 The Journal 114
B.9 OpenVMS BACKUP Savesets 115

Appendix C. The Stand-Alone Extractor vbkx **116**
C.1 Description 116
C.2 Format 117
C.3 Commands 118
C.4 Options 119
C.5 What Is Restored 120
C.6 Diagnostics 121
C.7 Completion Codes 122
C.8 vbkx.exe on Windows 123
C.9 The Extractors of Last Resort: vbkx-go, vbkx-rs, vbkx-pl 124

Appendix D. File Manager Plugins **126**
D.1 General 126
D.2 Midnight Commander 127
D.3 far2l and Far Manager 3 128
D.4 Total Commander and Double Commander 129

Preface

Intended Audience

This manual is for the system manager and the operator who save and restore the files of Linux systems. It assumes a working knowledge of Linux file systems, block devices, file ownership and permissions, and of the shell. Familiarity with the BACKUP utility of OpenVMS is helpful but not required.

The reader who wants to know what the bytes of a saveset are wants `doc/format.md` instead; this manual gives only a summary of them (Appendix B).

Why VBACKUP Exists

For forty years the BACKUP utility of OpenVMS has shown how a backup should be made: one command, two parameters, a qualifier for every choice, a checksum on every block, a group of blocks that repairs a bad one, a message for every condition. A tape written by it in one decade was read in the next. The machines it ran on are leaving the machine rooms; that way of working should not leave with them.

VBACKUP brings it to Linux. Its savesets are its own - made for the disks, the file systems and the attributes of Linux - but its command is that of BACKUP, its messages have the OpenVMS form and severities, it has a HELP library, its versions are IDENTs, and this manual is laid out like the manuals of OpenVMS. It also reads the savesets of OpenVMS BACKUP, so that what was saved on OpenVMS can be taken back on Linux.

Document Structure

This manual contains the following chapters and appendixes:

Part	Contents
Chapter 1	Introduces VBACKUP: what it does, what a saveset gives, what it does not do, the programs of the kit
Chapter 2	Installing VBACKUP on Linux and on Windows
Chapter 3	Describes the saveset and the mechanisms of VBACKUP: the structure of a saveset, file specifications, incremental backup and the journal, `/PHYSICAL` and `/IMAGE`, compression, encryption, pipes, savesets on other nodes, the page cache, threads, the extractors, the plugins, scheduling, the savesets of OpenVMS BACKUP, VBACKUP on Windows
Chapter 4	Contains examples of VBACKUP operations
Chapter 5	Contains the command dictionary: the `vbackup` command, its parameters, how the operation is determined, the exit status, the restrictions, and every qualifier
Chapter 6	Describes the format of a message and lists every message with what to do about it
Chapter 7	Troubleshooting: a symptom, what to look at, and what to do
Appendix A	Lists the files installed and used
Appendix B	Summarizes the saveset format
Appendix C	Describes the stand-alone extractor vbkx and the extractors of last resort
Appendix D	Describes the file manager plugins

The reader who installs VBACKUP for the first time wants Chapters 1, 2 and 4, in that order. The reader who holds a message wants Chapter 6, and the one who holds a symptom, Chapter 7.

Related Documents

- *vBACKUP saveset format, version 1* (`doc/format.md`) - the authoritative definition of the bytes of a saveset and of the journal. Where this manual and that document differ on the bytes of the medium, `format.md` prevails.
- *OpenVMS BACKUP savesets, as VBACKUP reads them* (`doc/vmsbackup.md`).
- *vBACKUP design document* (`doc/DESIGN.md`, in Russian) - the reasons behind the format and the program.
- The VBACKUP help library: `vbackup /HELP [topic ...]`; its sources are `doc/vbackuplib.md` (English) and `doc/vbackuplib_ru.md` (Russian).
- The manual pages `vbackup(1)` and `vbkx(1)`.
- The simple guides, task by task: `doc/simple/vbackup-en.md`, `doc/simple/vbackup-ru.md`, `doc/simple/vbackup-es.md`.
- The heads of the sources `tools/go/main.go`, `tools/rust/src/main.rs`, `tools/perl/vbkx.pl` and `plugins/wcx/vbkwcx.c`, which are the manuals of those programs.
- *OpenVMS System Management Utilities Reference Manual*, the chapter on BACKUP - the utility VBACKUP follows.

Acknowledgments

The authors thank Claude, the AI model of Anthropic, which went through the format, the code, the tests, the measurements and these manuals together with them.

The authors thank their comrades-in-arms of the NaPi Lab community, who paid for the subscription to Claude, so that an engineer of almost pensionable age could carry on this work.

License

VBACKUP is free software under the Apache License, Version 2.0: it may be used, copied, changed and given to others, also as part of other software and for money, provided that a copy of the License and the file `NOTICE` go with it, and that a changed file says that it was changed. The terms are in the file `LICENSE`, the notices in the file `NOTICE`; both are installed with the documents (Appendix A).

The images for Windows (`vbackup.exe`, `vbkx.exe`, the WCX plugin) carry, besides VBACKUP:

Part	License
StarLet (`utility_routines`), compiled into `vbackup.exe`	Apache License 2.0
winpthreads of MinGW-w64, linked statically	the expat license (mingw-w64 project) and the BSD 3-clause license (Lockless Inc.)

The texts of these licenses are in `win/COPYING.MinGW-w64`, given with every copy of the images. The compressors (LZ4 block, Deflate, LZMA) and the cryptography (ChaCha20, HMAC-SHA256, PBKDF2) are implementations of their own, written from the published descriptions; no code of other projects is included.

Trademarks

OpenVMS, VMS, Alpha, Integrity, Digital and DEC are trademarks of Hewlett Packard Enterprise Development LP or of VMS Software, Inc. Linux is the registered trademark of Linus Torvalds in the U.S. and other countries. Microsoft and Windows are trademarks of the Microsoft group of companies. Total Commander is a trademark of Christian Ghisler. Far Manager, Double Commander, Midnight Commander, far2l and Perl are the names of their projects. Go is a trademark of Google LLC. Rust is a trademark of the Rust Foundation. Arm and Cortex are trademarks of Arm Limited. Debian is a registered trademark of Software in the Public Interest, Inc. GitHub is a trademark of GitHub, Inc. NaPi is the name of the boards of the NaPi Lab team. Claude is a trademark of Anthropic, PBC. All other names of products and companies are trademarks of their respective owners.

VBACKUP is an independent implementation. It contains no code of OpenVMS and is not affiliated with, endorsed or supported by any of the owners named here; their names are used only to say what VBACKUP works with or follows. VBACKUP is not related to the Debian package `vbackup` nor to the VBackup program for Windows, which are other products of the same name.

Conventions

The following conventions are used in this manual:

Convention	Meaning
`UPPERCASE`	Uppercase words in a format are qualifier names and keywords. They may be typed in uppercase or lowercase and abbreviated as long as the abbreviation is unique.
italic	Italic words in a format and in message texts are placeholders for a value that you supply or that the system displays.
`[ ]`	In a format, square brackets enclose an optional part. Do not type the brackets. In `/[NO]VERIFY` they show that the qualifier may be negated.
`( , )`	In a format, parentheses enclose a list of values separated by commas. The parentheses are required when more than one value is given.
`...`	In a format, an ellipsis indicates that the preceding item may be repeated. In a file specification, `...` is the wildcard for any number of directory levels.
`monospace`	Monospace type shows what you type and what the system displays.
`$`	The prompt of an ordinary user's shell.
`#`	The prompt of the root shell: the command needs root privilege, or root is assumed.
Ctrl/*x*	Hold down the Ctrl key and press the key *x*.
`%VBACKUP-s-IDENT`	A message of the VBACKUP facility; *s* is its severity (S, I, W, E, F).

Every message the utility writes begins with the date, the time and the process number:

```
05-10-2026 14:19:25.787 2214682 %VBACKUP-I-STARTED, Operation: save, Input: /home/rrl, Output: rrl.bck -
started
```

Note

> In the examples of this manual the date, time and process number are omitted, and long directory names of the system on which the examples were made are shortened to names such as `/home`, `/backup` and `/mnt/usb`. Numbers and texts are as VBACKUP X01-11 and later print them.

Numbers are decimal unless otherwise stated. Sizes with the suffixes K, M, G and T are binary: 1K is 1024 bytes.

Chapter 1. Introduction

1.1 What VBACKUP Does

VBACKUP saves the files and directories of Linux file systems into a *saveset*, restores them from it, lists it and compares it with the disk. It also copies files from disk to disk, copies a whole block device block by block, saves and makes again a whole file system, and keeps a journal for incremental backup. It follows the BACKUP utility of OpenVMS: one command, two parameters, and the operation is determined by what the parameters are.

```
$ vbackup /home/rrl /backup/rrl.bck /VERIFY
$ vbackup /backup/rrl.bck /LIST
$ vbackup /backup/rrl.bck /tmp/restore
```

The first command saves the directory `/home/rrl` and checks the saveset against the disk; the second lists it; the third restores it into `/tmp/restore`.

1.2 What a Saveset Gives

- The files with their owners, permissions, times to the nanosecond, extended attributes, POSIX ACLs, capabilities and chattr flags; hard links, symbolic links, sparse files, FIFOs and devices.
- A checksum of every block and of every file: damage is always found.
- XOR blocks (`/GROUP_SIZE`): one bad block in a group is rebuilt; with `/PARITY=`*m* (Reed-Solomon) any *m* of them. After a loss the reader picks the stream up again at the next good block, and only the files that lay in the lost blocks are hurt.
- Volumes (`/VOLUME_SIZE=4G`), each of which identifies itself - on disk, through a pipe, or straight onto another node (`vbackup /home host::/backup/home.bck`).
- A catalog at the end: `/LIST` and `/EXTRACT` are instant.
- Incremental saves: `/RECORD` keeps a journal, `/SINCE=BACKUP` saves what changed since, and a chain restored `/INCREMENTAL` comes back with the deleted files gone.
- Compression on all cores: LZ4, Deflate or LZMA, small files packed together (SOLID), every record checked before it is written.
- Encryption (`/ENCRYPT`): ChaCha20 and HMAC-SHA256 per block, keys by PBKDF2; the names of the files are hidden too, and damage is still repaired without the passphrase.
- A saveset that can be read where VBACKUP is not installed: by `vbkx`, linked statically, and by three extractors of last resort in Go, Rust and Perl.

1.3 What VBACKUP Does Not Do

- It does not schedule and does not rotate savesets. That is the work of a batch queue - BATCH or cron - which runs a command procedure that runs VBACKUP (Section 3.15).
- It never changes a saveset in place: nothing is added to an existing saveset or deleted from it.
- It reads the savesets of OpenVMS BACKUP but does not write them, and does not read those encrypted by BACKUP (DES) nor their `/IMAGE` data as a volume (Section 3.16).
- It is not a store of deduplicated chunks: every saveset holds all its data, and can be read alone.
- It does not freeze a file system. A file that changes while it is saved is reported (FILCHANGED); for a consistent copy of a busy system save a snapshot of it.

1.4 The Programs of the Kit

Program	What it is
`vbackup`	The utility: every operation of this manual
`vbkx`	The stand-alone extractor: lists, extracts and tests a saveset; linked statically (Appendix C)
`vbkx-go`, `vbkx-rs`, `vbkx-pl`	The extractors of last resort, in Go, Rust and Perl, built from the sources in `tools/` (Appendix C.9)
`vbackup.exe`, `vbkx.exe`	The utility and the extractor on Windows (Section 3.17)
Plugins	Midnight Commander, far2l and Far Manager, Total Commander and Double Commander open a saveset as an archive (Appendix D)

Chapter 2. Installation

2.1 Requirements

Item	Requirement
Operating system	Linux, x86_64 or aarch64
Compiler	A C11 compiler (GCC)
Build	CMake 3.16 or later, make
Libraries	StarLet 1.6 or later (required); HELP 1.0 or later (optional: `/HELP` and the help libraries)
Optional	Go and Rust for `vbkx-go` and `vbkx-rs`; MinGW-w64 and wine for `vbkx.exe` and `vbackup.exe`; Midnight Commander or far2l for their plugins

Nothing else is needed at run time beyond the C library: VBACKUP links no compression nor crypto library.

2.2 Building and Installing

Install StarLet (and HELP) first, from their kits. Then:

```
# tar xzf vbackup-X01-25.tar.gz
# cd vbackup-X01-25
# cmake -S . -B build
# cmake --build build -j
# (cd build && ctest)
# cmake --install build
```

Every test must pass; a test that fails is a reason not to install. The tests speak TAP when `TAP=1` is set, for prove and the CI systems.

The tests make their scratch trees under the build tree. When the build tree is on a `/tmp` of tmpfs - small on a small board, and without user xattrs - give `TMPDIR` a real file system at the time of the configuration, and the scratch trees go there (or name the place with `-DVBACKUP_TESTDIR=`):

```
# TMPDIR=/var/tmp cmake -S . -B build
```

`make kit` in the build tree makes `vbackup-`*ident*`.tar.gz`; in a git tree it is made by git archive of HEAD, so only what is committed goes into it.

The options of the build:

Option	Default	Effect
`VBACKUP$M_HELPAPI`	ON	Answer `/HELP` out of the help library when HELP is found
`VBACKUP$M_HELPLIB`	ON	Translate and install the help libraries
`VBACKUP$M_INSTALL_DOC`	ON	Install the manual pages and the documents
`VBACKUP$M_PLUGINS`	ON	Install the plugins where Midnight Commander and far2l are found
`VBACKUP$M_VBKXSTATIC`	ON	Link vbkx statically when the system allows it
`VBACKUP$M_GEEKS`	ON	Build vbkx-go and vbkx-rs when Go and Rust are found
`VBACKUP$M_WINVBKX`	ON	Build vbkx.exe with MinGW-w64 and test it under wine, when they are found
`VBACKUP$M_PDF`	OFF	Render this manual as PDF (needs pandoc and XeLaTeX)
`VBACKUP$M_STRICT`	OFF	Treat the compilation diagnostics as errors

2.3 Installing on Windows

The images for Windows are made by GitHub Actions from every tag and published with the release: `vbackup-`*tag*`-win64.zip` holds `vbackup.exe`, `vbkx.exe`, the Total Commander plugin (`vbackup.wcx64`, `vbackup.wcx`, `pluginst.inf`), `LICENSE`, `NOTICE` and `COPYING.MinGW-w64`, and `SHA256SUMS` lists the checksums. Unpack the archive into a directory on the PATH; nothing else is installed. `tools/mkrelease.sh` makes the same files by hand.

2.4 Installing in Another Place

The default prefix is `/usr/local`. Another is given to CMake:

```
# cmake -S . -B build -DCMAKE_INSTALL_PREFIX=/opt/vbackup
```

2.5 Verifying the Installation

```
$ vbackup /HELP
$ vbackup /usr/local/share/doc/vbackup /tmp/doc.bck /VERIFY
$ vbkx t /tmp/doc.bck
```

The first command shows the topics of the help library; built without HELP, VBACKUP shows the summary built into the image instead. The second saves the installed documents and compares the saveset with the disk: it must end with `%VBACKUP-I-COMPLETED` and the exit status 0. The third tests the same saveset with the stand-alone extractor.

2.6 Removal

```
# cmake --build build --target uninstall
```

removes every file the installation put in place, and takes the sections of the plugins out of the files of Midnight Commander and far2l exactly, between their markers. Savesets and journals are never touched.

Chapter 3. Concepts

The VBACKUP utility (VBACKUP) saves files and directories of Linux file systems into a *saveset*, restores them from it, lists it and compares it with the disk. It also copies files from disk to disk, copies a whole block device block by block, saves and makes again a whole file system, and keeps a journal for incremental backup. VBACKUP follows the BACKUP utility of OpenVMS: one command, two parameters, and the operation is determined by what the parameters are.

VBACKUP needs nothing on the system beyond the C library; savesets can be read without VBACKUP by the stand-alone extractor vbkx and by the extractors of last resort (Appendix C).

3.1 Savesets

A saveset is a file, or a set of files called *volumes*, that holds copies of files together with everything needed to give them back:

- the data of regular files, sparse files with their holes;
- directories, symbolic links, hard links, FIFOs, character and block device files;
- the owner and group, by number and by name;
- the mode, including the setuid, setgid and sticky bits;
- the modification, access, change and creation times, with nanoseconds;
- extended attributes, POSIX ACLs, file capabilities and the SELinux label;
- the file system flags of `chattr` (FS_IOC_GETFLAGS).

Besides the files, a saveset holds a summary of the save (the product, the node, the user, the command, the time, the bases of the input specifications, the block and group sizes, the operating system), a checksum of every block and of every file, XOR blocks for repair, and a catalog of its files.

VBACKUP recognizes a saveset on the input side by its contents, never by its name. On the output side of a save, a saveset is recognized by its name (it ends in `.bck` or `.sav`), by the `/SAVE_SET` qualifier, or by the name `-` (the standard output). A saveset on another node, *node*`::`*file*, is a saveset on either side (Section 3.10).

3.2 Operations

The operation is determined by the parameters and by a few qualifiers (see Section 5.1):

Input	Output	Operation
files	saveset	**Save**: the files are written into a new saveset.
saveset[,...]	directory	**Restore**: the files of the saveset are created in the directory.
saveset	- (with `/ORIGINAL`)	**Restore** of every file into the directory it was saved from.
saveset	- (with `/LIST`)	**List**: the summary and the files of the saveset are displayed.
saveset	[directory] (with `/COMPARE`)	**Compare**: every file of the saveset is compared with the disk.
saveset	[file] (with `/EXTRACT`)	**Extract**: one file is written to a file or to the standard output.
files	directory (no saveset)	**Copy**: the files are copied from disk to disk.
saveset[,...]	- (with `/RECORD`)	**Journal rebuild**: the journal is made again from the catalogs.
-	- (with `/JOURNAL /LIST`)	**Journal listing**: the savesets and files the journal knows.
saveset	saveset	**Copy of a saveset**: the blocks are copied as they are, volume for volume (`/TRANSFER`).

A save may also verify the new saveset (`/VERIFY`), list it (`/LIST`), record it in the journal (`/RECORD`) and delete the files it saved and verified (`/DELETE`).

The saveset of a save, a restore, a listing, a compare, an extraction or a copy of a saveset may also be `-`, a pipe (Section 3.9), or *node*`::`*file*, a saveset on another node (Section 3.10).

3.3 Saveset Structure

Blocks

A saveset is a sequence of blocks of one fixed size, the *block size* (`/BLOCK_SIZE`, 8192 to 1048576 bytes, a multiple of 512; default 65536). Each block begins with a 64-byte header that carries the identifier of the saveset (a random UUID), the block number, the volume number, the block type and a CRC-32 of the header and the payload. The blocks are numbered from 0 continuously through all volumes. The first block of every volume is a *volume header* (VHDR) that holds the summary of the saveset, so that every volume identifies itself and the saveset it belongs to.

The payloads of the data blocks, taken in order, form one stream of records: a SUMMARY record, then for every file a FILE record, its DATA (or compressed DATAZ) records and an FEND record carrying the size, the CRC of the data and the status of the file; then the CATALOG records and an END record. A record may cross block and volume boundaries.

CRC

Every block carries a CRC-32 (IEEE) of its header and payload; every file carries a CRC-32 of its data in its FEND record and catalog entry. A damaged block is always detected when the saveset is read; a file whose data does not match its CRC is reported (CRCERR), never silently restored as good.

XOR Groups and Repair

After every *n* data blocks (*n* is the *group size*, `/GROUP_SIZE`, 0 to 100, default 10) VBACKUP writes one XOR block: the byte-wise XOR of the payloads of the data blocks of the group. When the saveset is read and exactly one data block of a group is bad, it is rebuilt from the other blocks of the group and the XOR block, and the message BLKFIXED is displayed; nothing is lost. When two or more blocks of a group are bad, they are reported (BLKLOST); the reader resumes at the first record that begins in the next good block, and only the files whose data lay in the lost blocks are affected (FILDAMAGED, FILLOST). `/GROUP_SIZE=0` writes no XOR blocks, and nothing can be repaired. The XOR blocks of the default group size make the saveset about 10% larger.

Volumes and Their Naming

`/VOLUME_SIZE` cuts the saveset into volumes of a fixed size. Volume 1 has the name given in the command; volume *k* (*k* >= 2) has that name followed by a period and *k* in decimal, at least three digits:

```
home.bck  home.bck.002  home.bck.003  ...
```

The volume size is rounded down to a multiple of the block size; every volume except the last has exactly that size. A group of blocks never crosses a volume boundary. To read a saveset, give the name of its first volume; VBACKUP finds the others beside it. A missing volume is reported (MISSVOL); the files that lie wholly in the other volumes are still restored. A file with a volume's name that belongs to another saveset is ignored (WRONGVOL).

A saveset written to a pipe carries its volumes back to back, each beginning with its VHDR, the block numbers running on through them (Section 3.9); a copy of the saveset into files splits it into volume files of the names above.

The Catalog

At the end of the stream VBACKUP writes a catalog: one entry for every file, with its name, type, owner, mode, size, times, link target, CRC, status and the place (volume, block, offset)

where its FILE record begins. `/LIST` reads the catalog only, so even a very large saveset is listed at once; `/EXTRACT` reaches one file through its place in the catalog without reading anything else. The catalog of an incremental saveset lists also the files that it covers but did not save, with the status PRESENT (see Section 3.5).

Nothing left out without a word. Since X01-22 a restore, `/COMPARE`, `/VERIFY` and vbkx `x` and `t` always set the catalog against the FILE records they met - not only after lost blocks. A file of the catalog whose records never came is named: FILLOST when blocks were lost, NOTINSTREAM when none was (a record the reader does not know, a saveset made wrong); `/COMPARE` and `/VERIFY` count it as a difference (COMPARERR). From a pipe the catalog records are taken as they come.

The TRAILER

The last block of the saveset is the TRAILER. It holds the totals of the save (files, bytes, errors, blocks, volumes, catalog entries) and the place where the catalog begins. A saveset without a TRAILER - the save was interrupted, the disk became full, or the last volume is missing – can still be read sequentially up to its last good block; VBACKUP reports NOTRAILER, and `/LIST` reads the whole saveset (NOCATALOG).

3.4 File Specifications and Wildcards

An input specification of a save or a copy is a file or directory name, optionally with wildcards. VBACKUP expands the wildcards itself; put them in quotes so that the shell does not:

Wildcard	Matches
`*`	any characters within one name component
`% ?`	exactly one character
`...`	any number of directory levels, none included (as a whole component)

The *base* of a specification is the part before the first component that holds a wildcard or `...`; without wildcards, the base is the directory that contains the named file or directory. The names stored in the saveset - the *stored names* - are relative to the base. A directory named in a specification is therefore stored with its own name in front:

Specification	Base	Stored names
`/home/rrl`	`/home`	`rrl`, `rrl/...`
`/home/rrl/src/.../*.c`	`/home/rrl/src`	`a.c`, `lib/b.c`
`/etc/*.conf`	`/etc`	`host.conf`, ...
`/mnt/data/.`	`/mnt/data/.`	the contents of `/mnt/data`, under their own names

A directory selected by a specification is taken with everything in it. Hidden files (names beginning with a period) are matched. The entries of a directory are walked in the byte order of their names, so two saves of the same tree produce the same saveset layout. The bases are recorded in the summary as absolute names (realpath); `/ORIGINAL` and `/COMPARE` without an output use them.

The patterns of `/SELECT` and `/EXCLUDE` are matched against the stored names. In a pattern, `*` matches any characters including `/`, and `%` and `?` match one character.

A word of the command that begins with a slash is a qualifier only when it names one; `/etc` and `/home` are file names. A specification that contains a wildcard is never taken for a saveset.

3.5 Incremental Backup and the Journal

VBACKUP distinguishes two sets of files in a saveset (`format.md`, 6.6):

- the *covered* set: every entry selected by the input specifications and the name filters - `/SELECT`, `/EXCLUDE`, the nodump flag, `/NOCROSS_DEVICE` - and by `/BY_OWNER`;
- the *saved* set: the part of the covered set whose data is in the saveset. In a *full* saveset it is the covered set itself. In an *incremental* saveset the time filters - `/SINCE`, `/BEFORE`, `/SINCE=BACKUP` - choose it.

Any time filter makes the saveset incremental. Its catalog lists the whole covered set: the saved files with their status, the others as PRESENT. The catalog thus describes the tree as it was at the time of the save, and a restore `/INCREMENTAL` can remove from a directory what was no longer in it. Directories are always saved.

The *journal* is a file of its own, outside any saveset. A save with `/RECORD` records in it the saveset and, for every file saved with the status OK, its absolute name, inode, size, modification and change times. A save with `/SINCE=BACKUP` saves a file when the journal does not know it or when its inode, change time, modification time or size differ from the recorded state; the device number is recorded but not compared. A file saved CHANGED or with a read error is not recorded, so the next `/SINCE=BACKUP` saves it again. Under `/VERIFY` the journal is updated only when the verification found no difference.

The journal is `/var/lib/vbackup/vbackup.jnl` for root and `~/.vbackup/vbackup.jnl` for other users; `/JOURNAL=file` names another. It is rewritten whole under an exclusive lock (`journal.lock`), through `journal.tmp`. A lost or damaged journal can be rebuilt from the catalogs of the savesets (`saveset[,...] /RECORD`); the journal is a convenience, never the only copy of anything.

A chain - the full saveset, then the incremental ones in the order they were made - is restored with `/INCREMENTAL`. Note that OpenVMS BACKUP restores an incremental chain newest first; VBACKUP takes it oldest first.

3.6 Physical and Image Operations

/PHYSICAL

A save `/PHYSICAL` copies every block of one block device - a partition or a whole disk - or of an image file, whatever it holds: any file system, an encrypted volume, boot areas. The saveset holds one regular file, the image of the device, named after the device node; pieces of 65536 bytes that are all zeros are not written. VBACKUP refuses a device that is mounted read-write (or a partition of which is), and a device in use (held by LVM, RAID, dm-crypt, or used as swap). A device mounted read-only is accepted.

A restore `/PHYSICAL` writes the image onto a device or into an image file. Onto a device it requires `/REPLACE`, refuses a mounted or busy device and a device smaller than the one saved, writes zeros into the gaps, and at a terminal asks for `YES`. Onto a larger device the first part is written and the rest is left as it is. The copy carries the labels and UUIDs of the original (PHYSUUID): never mount both at the same time.

A `/PHYSICAL` saveset restored without `/PHYSICAL` is a sparse image file in the output directory; every extractor restores it so.

/IMAGE

A save `/IMAGE` saves every file of one mounted file system - nothing of the file systems mounted below it, nodump flags ignored - under names relative to its root, and records what makes the volume that volume: its type, label, UUID, space in use, mount options and the owner, mode, times and ACLs of its root directory. The input is the mount point, or the device when it is mounted (read-only is enough).

A restore `/IMAGE` onto a device makes a new file system of the saved type, label and UUID by running `mkfs.ext2`, `mkfs.ext3`, `mkfs.ext4`, `mkfs.xfs`, `mkfs.btrfs` or `mkfs.vfat` (that program must be installed), mounts it on a temporary directory with `nosuid,nodev,noexec`, restores the files into it, gives its root the saved attributes and unmounts it. The guards are those of `/PHYSICAL`; the device must hold the space in use plus 5% plus 16 MB, and may be smaller or larger than the one saved. `/IMAGE` copies neither the boot sectors nor the partition table nor the inode numbers; for those use `/PHYSICAL` of the whole disk.

An `/IMAGE` saveset restored without `/IMAGE` is an ordinary tree of files.

3.7 Compression

`/DATA_FORMAT=COMPRESSED` stores the data of files in DATAZ records; `/LEVEL=n` says how hard, and compresses by itself:

/LEVEL	Codec	Text (1 MB record)	Binaries	Save, one core	Restore, one core
1 (default)	LZ4	2.1x	1.5x	130 MB/s	400 MB/s
2 .. 5	Deflate	3.2x .. 3.7x	1.9x .. 2.1x	33 .. 11 MB/s	110 .. 230 MB/s
6, 7	LZMA, fast parse	4.0x .. 4.2x	2.3x	12 .. 8 MB/s	30 .. 60 MB/s
8, 9	LZMA, optimal parse	4.8x .. 4.85x	2.4x	1.9 .. 1.7 MB/s	30 .. 65 MB/s

The ratios were measured on this host on 1 MB records of the vBACKUP source tree and of program binaries; the speeds are those of one core, the check of every record included - the compression runs on several threads (Section 3.12), and the saveset does not depend on their number. A restore is fast at every level: faster than most disks for LZ4 and Deflate, tens of MB/s for LZMA. Levels 8 and 9 choose the cheapest coding of every stretch of data by the prices of the model (the optimal parse of xz) and compress as `xz -6` does on a record of 1 MB; they are slow to save, as xz is, and as fast to restore as 6 and 7.

Small Files: SOLID Records

Small files compress badly one by one: each starts with an empty dictionary, and its FILE and FEND records are not compressed at all. Since X01-21 a compressed save therefore gathers the small regular files – each of at most 256 KB, not a further name of a hard link - into groups of up to 256 KB (1 MB in X01-21), in the order of the walk, and compresses each group as one SOLID record (format.md 6.12): the FILE, DATA and FEND records of its files together, so the dictionary is shared and the metadata is compressed too. A directory, a link or a bigger file closes the open group first. It is the default whenever the save compresses; `/NOSOLID` writes every file in records of its own, as before. Measured on this host (saveset sizes):

Tree	/LEVEL	/NOSOLID	/SOLID, X01-21 (1 MB groups)	/SOLID, X01-22 (256 KB, default)
`/usr/include` (210 MB, 22 444 files)	1	81 MB	62 MB	62 MB
`/usr/include`	2	60 MB	44 MB	45 MB
`/usr/include`	5	55 MB	39 MB	39 MB
`/usr/include`	6	56 MB	36 MB	37 MB
`/usr/include`	9	53 MB	32 MB	33 MB
`/usr/lib/python3` (512 MB)	1	269 MB	254 MB	255 MB
`/usr/lib/python3`	2	200 MB	186 MB	187 MB
`/usr/lib/python3`	5	187 MB	172 MB	173 MB
`/usr/lib/python3`	6	172 MB	146 MB	152 MB
`/usr/lib/python3`	9	163 MB	135 MB	141 MB

The speed of the save and of the restore is about the same either way. Big files are compressed record by record as before. X01-22 makes the groups four times smaller than X01-21 did: the saveset is a few per cent bigger, and far less is lost on a bad day (below).

What it costs on the day of the restore. With the default parity nothing changes: one bad block of a group is still rebuilt (any *m* with `/PARITY=m`). But blocks lost beyond repair - more than *m* bad blocks in one group - take more than the file they were in: the files of the SOLID they cut from the damage on, and every file of a SOLID that begins in them. The part of a SOLID before the cut is not lost (since X01-22): it is decompressed as far as it is exactly right, and the files wholly in it are restored, each checked by its FEND CRC; the file at the cut is restored as far as it came (FILDAMAGED), the files after it are named with FILLOST. The same holds for the end of a saveset whose save was interrupted. The other files are restored. Files lost per parity group damaged beyond repair, measured on `/usr/lib/python3/dist-packages`:

Saveset	/LEVEL=6	/LEVEL=1
`/NOSOLID`	19	11
`/SOLID`, X01-21 (1 MB groups, no salvage)	52	45
`/SOLID`, X01-22 (256 KB groups, salvage)	24	22

So a handful to a couple of dozen small files, not one. The salvage is the reader's: VBACKUP X01-22 does it for savesets of X01-21 too. For a saveset that must live through heavy damage, give `/PARITY=2` or more (see `/PARITY` in Section 5.2), or `/NOSOLID`.

A SOLID that does not decompress at all, or holds a record that makes no sense, is treated like lost blocks (BADREC, then FILLOST for its files) – never as wrong data. The writer decompresses every SOLID and compares it before it is written; a group that does not come back the same (ZCHECK) or does not shrink is written as ordinary records.

Records and Checks

Every DATA record of a big file is compressed on its own (format.md 6.7): a bad block costs that record alone, and the XOR and parity repair works as for uncompressed data - the file CRC is that of the raw bytes, the block CRC that of the bytes as they lie in the stream. For each record VBACKUP first tries the first 64 KB and stores the record uncompressed when it does not shrink by at least 3%; already compressed data (photographs, video, `.gz`, `.zip`) costs almost no time. A DATAZ record is written only when it is smaller than the DATA record would be.

Restorable on the day it is needed. Every record VBACKUP compresses is decompressed again at once and compared with the data; a record that does not come back the same is written uncompressed, and ZCHECK says so. The saveset never depends on the compressor being right. Each codec is read by four independent decoders - the core of VBACKUP (vbackup, vbkx, vbkx.exe, the WCX plugin), vbkx-go, vbkx-rs, vbkx-pl - and the formats are the public ones (raw Deflate of RFC 1951, raw LZMA1), described in `format.md` well enough to write a fifth; zlib and liblzma read them.

VBACKUP and vbkx before X01-04 cannot read compressed data, before X01-19 not levels 2 to 9; they report the files as damaged (CRCERR, FILDAMAGED, BADREC) and never write wrong data silently. A compressed saveset of X01-21 or later, unless saved with `/NOSOLID`, has format version 3 in every block - even when every group happened to be written as ordinary records; VBACKUP, vbkx and the plugins before X01-21 refuse it as a whole (NOTSAVESET) and never misread it. Read it with X01-21 or later, or save with `/NOSOLID` for older readers. The smaller groups of X01-22 are the same version 3: X01-21 reads them, and X01-22 reads the savesets of X01-21.

3.8 Encryption

`/ENCRYPT` makes an encrypted saveset (`format.md`, 6.10). The payloads of the data blocks and of the TRAILER are encrypted with ChaCha20 and authenticated with HMAC-SHA256, each block with a 32-byte tag. The keys are derived from a passphrase by PBKDF2-HMAC-SHA256, 600000 iterations by default, with a random 32-byte salt taken anew for every saveset. No cryptographic library is used: vbkx and the extractors of last resort read encrypted savesets with nothing else.

Keys and KEYCHECK

From the passphrase and the salt VBACKUP derives a master key and from it three keys: one for encryption, one for authentication, and a check value. The check value is stored in the volume header as KEYCHECK. A reader derives the keys and compares the check value before it reads anything else: a mismatch means a wrong passphrase (WRONGKEY), not a damaged saveset.

What Is Visible

Block headers, CRCs and XOR blocks are computed over the ciphertext, so a damaged encrypted saveset is checked and repaired without the passphrase. Without the passphrase the following remain visible: the block size, group size, volume size and the number of volumes; the length of the stream and where records begin in each block; the time of the save only through the file times of the volumes. Not visible: the names, sizes, attributes and contents of the files, the node, the user, the command and the totals of the TRAILER.

A block whose CRC is right but whose tag is wrong was changed on purpose (or by a very odd medium): it is reported (BLKFORGED) and treated as a bad block - repaired from its group when possible, otherwise lost.

Passphrase Handling

The passphrase is never taken from the command line: the command is stored in the saveset and is visible to other users through `ps`. It is obtained, in this order:

1. from the key file named by `/KEY_FILE=file`;
2. from the key file named by the environment variable `VBACKUP_KEY_FILE`;
3. from the terminal (`/dev/tty`), without echo; for a save it is asked twice, and the two must agree (otherwise KEYMATCH and nothing is saved).

When neither a key file nor a terminal is available (cron, a pipe, a file manager), or `VBACKUP_NOPROMPT=1` is set, VBACKUP stops with NOKEY; it never waits. The passphrase is asked once per command and kept for the savesets that follow (the verification of a save, the chain of an `/INCREMENTAL` restore); it is wiped from memory at exit and on Ctrl/C, which also restores the terminal echo.

Key Files

The passphrase is the first line of the key file, without its line end (LF or CR LF), taken byte for byte; it may be at most 1024 bytes and must not be empty. The key file must be a regular file readable and writable by its owner only (mode 600); otherwise it is refused (KEYFILE).

The passphrase *is* the key: a saveset whose passphrase is lost cannot be opened by anybody. Use a long passphrase - five or more random words. The journal is not encrypted; it is a file of the saving system.

Opening an encrypted saveset costs the PBKDF2 computation: about 0.5 s on a PC, 1.6 s on a small ARM board with SHA instructions, 6 s on one without. That cost is deliberate; every guess of an attacker pays it too.

VBACKUP and vbkx before X01-06 cannot read an encrypted saveset; they report all blocks lost and write nothing.

3.9 Pipes

The name - is the standard output as the output of a save or of a copy of a saveset, and the standard input as the input of a restore, /LIST, /COMPARE, /EXTRACT and a copy of a saveset:

```
$ vbackup /home - | ssh host 'vbackup - /backup/home.bck'
$ ssh host 'vbackup /home -' | vbackup - /restore
```

Streams of Several Volumes

Since X01-11 a saveset written to - may have several volumes (/VOLUME_SIZE): they go to the standard output back to back, each beginning with its VHDR, and the block numbers run on through them as through volume files. Every volume after the first is reported (CREATED, `Volume: (standard output)`). A reader of the stream takes the VHDR of the next volume as the end of the volume in hand; a volume missing from the stream is reported (MISSVOL), and the files that lie wholly in the other volumes are still restored.

A saveset written to - is gone once written: /VERIFY (hence /DELETE) and /LIST are refused with it (QUALUSE: `a saveset written to the standard output is gone once written: it cannot be read back here`).

A saveset read from - is read once, forward only. The catalog at its end is of no use there: /LIST lists the FILE records as they come, /EXTRACT finds its file by reading, and vbkx refuses names. Bad blocks are repaired as from a file. A pipe that ends before the TRAILER gives NOTRAILER.

The Receiver

When the input is a saveset and the output is a saveset too, VBACKUP copies the saveset block for block (/TRANSFER). With - on one side this joins a pipe and the volume files:

```
$ vbackup home.bck - | ...          the volumes, back to back, into the pipe
$ ... | vbackup - home.bck          the stream into home.bck, home.bck.002, ...
```

The receiver, `vbackup -`*saveset*, splits the stream into volume files at each VHDR, with the names of Section 3.3: the saveset on its disk is the one that was sent, volume for volume and byte for byte. It checks every block as it arrives. A bad block is copied as it is and reported (BLKCOPIED), to be repaired from its group by a restore; a stream that ends before its TRAILER is reported (NOTRAILER). The records are never decoded, so an encrypted saveset passes without its passphrase. XFRSUMM gives the totals.

3.10 Savesets on Other Nodes

A saveset may lie on another node, named as DECnet named a file there: *node*::*file*. *node* is what ssh takes (`host` or `user@host`) and contains no slash; *file* is the saveset (its first volume) on that node.

```
$ vbackup /home backup-host::/backup/home.bck /VOLUME_SIZE=4G /VERIFY
$ vbackup backup-host::/backup/home.bck /restore
$ vbackup backup-host::/backup/home.bck /LIST
```

Mechanism

VBACKUP starts VBACKUP on the other node through ssh - or through the command named by the environment variable `VBACKUP_RSH` - and joins it to this one by a pipe; the saveset goes through the pipe as a stream (Section 3.9). What is given to the other node is always a copy of a saveset:

This node	The other node
A save, or a copy of a saveset, to *node*::*file*: the saveset is written into the pipe.	`vbackup -`*file*`/TRANSFER`, with `/REPLACE` when it is given here: the stream is split into the volume files of *file*, every block checked.
A restore, `/LIST`, `/COMPARE`, `/EXTRACT` or a copy of a saveset from *node*::*file*: the saveset is read from the pipe.	`vbackup`*file*`- /TRANSFER`: the volumes of *file*, back to back, into the pipe.

`/VOLUME_SIZE` of a save to another node therefore makes the volume files there. A *node*::*file* input must be the only input. The messages of the other side come to the standard error here as they are, among those of this side and with its own process number: its STARTED, CREATED, XFRSUMM and COMPLETED, and whatever went wrong there.

/VERIFY

A save to another node with `/VERIFY` is verified once the other side has completed: the saveset is read back from there (a second ssh, `vbackup`*file*`- /TRANSFER`) and every file is compared with the disk here (VERIFYING, CMPSUMM). The saveset as it arrived - the CRC of every block, the TRAILER - is checked by the receiver there. `/DELETE` and `/LIST` are not taken with a saveset made on another node (QUALUSE: `not with a saveset made on another node`); list it by a second command, which reads it back. With `/RECORD` the journal records such a saveset as *node*::*file*.

Completion Codes

The completion code of the other side joins that of this one. 0 adds nothing; 1 (warnings) makes the exit status at least 1; any other code, or an end by a signal (shown as 128 + the signal), is reported by REMOTEERR, and the exit status is 2. Code 127 most often means that the command of `VBACKUP_RSH` cannot be run here, or `vbackup` cannot be found there; 255 is ssh itself failing. When the other side stops while this one writes, this side finds the pipe broken: WRITERR (errno 32), FATALSAVE and REMOTEERR follow one another - one fault, told by the messages of the other side above them. An `/EXTRACT` stops reading once it has its file; the broken pipe the other side meets then is no error. REMOTE reports that the pipe or the process for ssh cannot be made at all.

Requirements

- VBACKUP X01-11 or later on the other node, in the PATH of a non-interactive ssh session: `ssh`*node*`vbackup` must start it.

- ssh must let this node in by its keys, with nothing asked: the key of the user installed there (`ssh-copy-id`), the host key of the other node already known.
- The user on the other node must be able to create, or read, the saveset there.

Why /TRANSFER

The commands given to the other node carry `/TRANSFER`. A VBACKUP before X01-11 does not know that qualifier and refuses the command; without it, it would take `vbackup - /backup/home.bck` for a restore of the stream into a directory named `/backup/home.bck`. An old VBACKUP on the other node thus fails plainly (REMOTEERR) rather than does the wrong thing.

3.11 The Page Cache

VBACKUP takes care not to evict the working data of the system from the page cache:

- the saveset is written in windows of 8 MB: the writeback of each window is started at once, waited for one window later, and its pages are then dropped; every volume is synchronized when it is closed;
- the volumes of a saveset being read and the files being saved are read with sequential advice; a group of the saveset that has been read is dropped from the cache;
- a file being saved is dropped behind the read only when it was not in the cache before (no page of its first megabyte was resident); a file that was in the cache is somebody's working data and stays.

The pages of the saveset and of "cold" files are therefore not in the cache after a save; that is intended.

3.12 Threads

Thread(s)	Work	Number
Writer	Writes the blocks of the saveset, while the main thread reads files and builds blocks.	1; `VBACKUP_PIPELINE=0` writes without it.
Read-ahead	Opens and reads the first megabyte of the next files of the walk, so that the save or copy finds them in the cache; at most 128 files and 64 MB ahead.	8 by default; `VBACKUP_PREFETCH=n`, at most 64, 0 - none.
Compression	Compresses the data of a save `/DATA_FORMAT=COMPRESSED`; the main thread writes the records in order.	The number of processors, at most 8; `VBACKUP_ZTHREADS=n`, 1 - none.
Decompression	Restore, `/COMPARE` and `/VERIFY` (since X01-23): the records are read ahead, up to 4 a thread and 64 MB, the DATAZ and SOLID among them decompressed side by side, then handed out in their order.	The threads of the encryption: `VBACKUP_CTHREADS=n`, 1 - one record at a time.
Encryption	Writing: the encrypted data blocks waiting for the writer sealed (ChaCha20 and tag) side by side, each whole, while the writer thread computes the XOR blocks and the CRCs in order; without the writer thread, the key stream of a block in stripes. Reading: the tags of the blocks of a group checked and the blocks decrypted side by side.	The number of processors, at most 8; `VBACKUP_CTHREADS=n`, 1 - none.

No thread changes the saveset: with any number of threads the same input gives the same saveset.

3.13 The Stand-Alone Extractor and the Extractors of Last Resort

vbkx is a small, statically linked program that lists, extracts, prints and tests savesets on a machine where VBACKUP is not installed - a rescue system, another distribution. It needs no help library and no message facility. It restores data, holes, modes, times, symbolic and hard links and FIFOs, and as root the owners (by number) and device files; it does not restore ACLs, extended attributes or chattr flags. `vbkx.exe` is the same program for Windows.

Three more extractors - vbkx-go (Go), vbkx-rs (Rust) and vbkx-pl (Perl) – are written for the day everything else is dead: one source file each, the standard library only, plain so that they can be read and corrected against `format.md`. They read compressed and encrypted savesets, repair one bad block per group, and name every damaged or missing file. See Appendix C.

3.14 File Manager Plugins

Savesets can be browsed like folders in Midnight Commander (extfs script `uvbk`), far2l and Far Manager 3 (MultiArc, over vbkx), and Total Commander and Double Commander (WCX packer plugin). All plugins are read only: a saveset is never changed. The listing comes from the catalog, so even a saveset of hundreds of gigabytes opens at once. An encrypted saveset is opened with the key file named by `VBACKUP_KEY_FILE`; the plugins never ask for a passphrase. See Appendix D.

3.15 Scheduling and Rotation

VBACKUP does not schedule itself nor delete old savesets: that is the business of a batch queue, and BATCH (the batch job subsystem, `batch submit`) does it the way OpenVMS does - a job that runs the save and submits itself again for the next day.

```
#!/bin/sh
# /opt/jobs/vbackup-daily.sh - a full save on Sunday, an incremental on the other days
T=/backup/home-$(date +%Y%m%d)
if [ "$(date +%u)" = 7 ]; then
    vbackup /home $T.bck /RECORD /VERIFY
else
    vbackup /home $T.bck /SINCE=BACKUP /RECORD /VERIFY
fi
# Rotation: five weeks of savesets are kept
find /backup -name 'home-*.bck*' -mtime +35 -delete
# Again tomorrow at 02:00 - the job submits itself
batch submit "$0" /NAME=vbackup-daily "/AFTER=$(date -d tomorrow +%d-%b-%Y:02:00 | tr a-z A-Z)"
```

```
$ batch submit /opt/jobs/vbackup-daily.sh /NAME=vbackup-daily /AFTER=TOMORROW
```

A job that fails stays in the queue with its log (`batch show`), the messages of VBACKUP and its completion code in it; nothing is lost silently, as with cron.

3.16 OpenVMS BACKUP Savesets

A saveset written by OpenVMS BACKUP is accepted as input. It is known by the header of its first block (there is no magic string) and read from its first block to its last; VBACKUP displays VMSSAVESET with its block and group size. Copy it from VMS in binary mode (FTP `binary`); a saveset taken off a tape must keep its block size.

```
$ vbackup USERS.BCK /LIST /FULL
$ vbackup USERS.BCK /restore/vms /LOG
$ vbackup USERS.BCK "/EXTRACT=[SMITH]LOGIN.COM;3" login.com
```

`/LIST` and `/LIST /FULL` display what `BACKUP/LIST` and `BACKUP/LIST/FULL` display, octet for octet. A restore puts the files under the output directory by their Linux names: `[SMITH.WORK]NOTES.TXT;5` becomes `SMITH/WORK/NOTES.TXT`; the highest version takes the plain name, older versions keep `;n`; the escapes of ODS-5 are undone and 8-bit names made UTF-8; a directory file becomes the directory. `/SELECT`, `/EXCLUDE` and `/EXTRACT` take the Linux names; `/EXTRACT` also the name with its version or the file specification of VMS.

Sequential files with a carriage control become texts with LF, as FTP of VMS makes them: variable and VFC records lose their counts and control areas, fixed records are followed by LF, Stream turns CR LF into LF and Stream_CR CR into LF. Files without a carriage control are copied as they are on the VMS disk; indexed and relative files as the image of the RMS file (VMSRAW). The mode comes from the protection (R - r, W - w, E - x for directories only), the modification time from the revision date taken as local time; the owner is the user who restores.

Bad blocks are rebuilt from the XOR blocks of `/GROUP_SIZE` (BLKFIXED); blocks that cannot be rebuilt are reported (BLKLOST), and a file that lost data is reported (FILDAMAGED). A saveset written `/NOCRC` is reported once (VMSNOCRC). Savesets encrypted by BACKUP, and the LBN data of `/IMAGE` and `/PHYSICAL` savesets, are not read. `/COMPARE`, `/INCREMENTAL`, `/ORIGINAL`, `/IMAGE`, `/PHYSICAL` and `/TRANSFER` are not valid with such a saveset. The stand-alone extractor `vbkx` reads it the same way. The format, as VBACKUP reads it, is described in `doc/vmsbackup.md`.

3.17 VBACKUP on Windows

`vbackup.exe` is VBACKUP built for Windows with MinGW-w64: the same command, the same qualifiers, the same savesets. A saveset made on Linux is listed, restored and compared on Windows; one made on Windows is read on Linux, by `vbkx` and by every extractor of last resort. It is built on Linux from the same tree, with the StarLet sources (a StarLet installed for Linux does not serve):

```
$ cmake -S . -B build-win -DCMAKE_TOOLCHAIN_FILE=cmake/mingw-w64.cmake \
        -DVBACKUP_STARLET_SRC=/root/Works/starlet-1.6.8
$ cmake --build build-win
```

The Linux build makes it by itself (`build/winexe/vbackup.exe`) when MinGW-w64 and the StarLet kit of the installed version (`../starlet-x.y.z` beside the source tree) are at hand, and `test/winutil.sh` runs it under wine. The image is static: copy `vbackup.exe` anywhere and run it.

```
C:\> vbackup C:\Users\ivan\Documents D:\docs.bck /LOG
C:\> vbackup D:\docs.bck /LIST /FULL
C:\> vbackup D:\docs.bck C:\restore
C:\> vbackup C:\data D:\data.bck /ENCRYPT /KEY_FILE=C:\Users\ivan\backup.key
```

What Is Saved and Restored

Names are Unicode - UTF-8 in the saveset, as on Linux - and paths may be of any length (the `\\?\` form is used where needed). `\` and `/` both separate the components of a specification - a `\` anywhere on the command line is taken for a separator, in `/SELECT` and `/COMMENT` too; the stored names use `/`.

Saved and put back:

- the data; a sparse file by its allocated ranges, restored with its holes (the file is made sparse when a hole is written);
- the modification, access and change times and the creation time (BTIME);
- the attributes of NTFS - read-only, hidden, system, archive, temporary, not indexed (tag WINATTR); `/LIST /FULL` shows them as `Windows: HIDDEN ARCHIVE`. The mode is made up from read-only: `0444` or `0644` for a file, `0755` for a directory;
- the security descriptor - owner, group, DACL, and the SACL when the image holds the security privilege (tag NTSD); the owner and the group are shown by their accounts, `DOMAIN\user` (UID and GID are 0);
- the alternate data streams, as the extended attributes `user.<stream>` (the form ntfs-3g gives them on Linux), up to 64 KB each; a longer one is reported (XATTRSKIP);
- directories, hard links (by the file ID and the volume serial number of NTFS, as by the inode and device on Linux), symbolic links and junctions (as symbolic links).

On Linux a saveset made on Windows restores as any other: the streams become `user.` attributes, the attributes of NTFS and the descriptor are left aside; every extractor reads it.

Privileges and the Owner

Run as an administrator (elevated), `vbackup.exe` enables the privileges of a backup operator - SeBackupPrivilege, SeRestorePrivilege, SeSecurityPrivilege, SeCreateSymbolicLinkPrivilege - when its account holds them. It then reads every file whatever its ACL, makes symbolic links, and takes `/OWNER=ORIGINAL` by default, as root on Linux: the security descriptor of each file is put back, its owner, group, DACL (protected or inheriting, as it was) and, with the security privilege, its SACL. Without the restore privilege the default is `/OWNER=DEFAULT`: the descriptor is not put back, and a restored file takes the ACL its directory gives it - a DACL from another machine could leave the user who restores without access to the files. `/OWNER=ORIGINAL` asks for the descriptor all the same; an owner that cannot be set is reported

(ATTRERR ... security). `/OWNER=`*user* and `/BY_OWNER` are refused on Windows (QUALUSE): a SID is no user number. A descriptor read from a saveset is checked before use - every SID and ACL in it must lie inside the item - and a forged one is refused (ATTRERR ... the descriptor in the saveset is not valid).

Names Windows Cannot Hold

A Linux saveset may hold names that Windows does not allow or reads as something else: a `:` (a stream of another file), `< > " | ? * \`, a control character, a trailing dot or blank (dropped by Windows - two names, one file), the names of devices (`CON`, `PRN`, `AUX`, `NUL`, `COM1`-`COM9`, `LPT1`-`LPT9`, with an extension too). Such a file is not created: OPENOUT names it with errno 22 and the reason `not a valid name on Windows`, and the restore goes on with the others. Two names that differ in case only (`README`, `readme`) are one file on Windows: the second is not restored - FILEEXISTS, and with `/REPLACE` OPENOUT (`a name that differs in case only is there`), so that it never replaces the first. A FIFO, a socket or a device is not made (UNSUPP). Of the extended attributes of a Linux saveset only the `user.` ones become streams; the others (`security.`, `trusted.`, `system.`) are skipped.

Differences from Linux

- A saveset on another node (`node::file`) goes through `ssh.exe`, the OpenSSH client of Windows 10 and 11; `VBACKUP_RSH` names another one.
- `/PHYSICAL` and `/IMAGE` are refused at once (QUALUSE).
- There is no read-ahead of files (`VBACKUP_PREFETCH` has no effect) and no shadow copy (VSS): a file another program holds open without sharing is reported (OPENIN) and left out; one it writes to is saved as it is read (FEND CHANGED when it changed meanwhile).
- The mode of a key file is not checked: its ACL says who reads it. Keep it in your profile, where only you and the administrators can.
- The journal is `%USERPROFILE%\.vbackup\vbackup.jnl` (HOME, when it is set).
- The passphrase is read from the console without echo, in UTF-8: a saveset encrypted on Linux opens on Windows with the same passphrase.

Chapter 4. Examples

1.
```
$ vbackup /home/rrl /backup/rrl.bck /VERIFY
 %VBACKUP-I-STARTED, Operation: save, Input: /home/rrl, Output: /backup/rrl.bck -
started
   %VBACKUP-I-SAVESUMM, Files: 6, Bytes: 108902, Blocks: 5, Volumes: 1 - saved
   %VBACKUP-I-VERIFYING, Saveset: /backup/rrl.bck - verifying
   %VBACKUP-I-CMPSUMM, Files: 6, Differences: 0 - compared
   %VBACKUP-I-COMPLETED, Operation: save, Seconds: 0.00 - completed
```

   A full save of the directory `/home/rrl` into the saveset `rrl.bck`. The saveset is recognized by its name. `/VERIFY` reads the new saveset back and compares every file with the disk; no difference was found, and the exit status is 0. The stored names begin with `rrl/`: the base is `/home`.

2.
```
$ vbackup /home full.bck /RECORD /JOURNAL=/backup/home.jnl
 %VBACKUP-I-STARTED, Operation: save, Input: /home, Output: full.bck - started
 %VBACKUP-I-SAVESUMM, Files: 7, Bytes: 3108910, Blocks: 55, Volumes: 1 - saved
 %VBACKUP-I-RECORDED, Files: 5, Journal: /backup/home.jnl - recorded
 %VBACKUP-I-COMPLETED, Operation: save, Seconds: 0.01 - completed
 $ vbackup /home mon.bck /SINCE=BACKUP /RECORD /JOURNAL=/backup/home.jnl
 %VBACKUP-I-STARTED, Operation: save, Input: /home, Output: mon.bck - started
 %VBACKUP-I-SAVESUMM, Files: 3, Bytes: 4, Blocks: 4, Volumes: 1 - saved
 %VBACKUP-I-INCRSUMM, Files: 4 - unchanged, listed as present, not saved
 %VBACKUP-I-RECORDED, Files: 1, Journal: /backup/home.jnl - recorded
 %VBACKUP-I-COMPLETED, Operation: save, Seconds: 0.00 - completed
 $ vbackup full.bck,mon.bck /restore /INCREMENTAL /LOG
%VBACKUP-I-STARTED, Operation: restore, Input: full.bck, Output: /restore - started
 %VBACKUP-I-RESTORED, File: /restore/rrl - restored
 ...
 %VBACKUP-I-RESTORED, File: /restore/rrl/new.txt - restored
 %VBACKUP-I-RESTORED, File: /restore/rrl/sub - restored
%VBACKUP-I-DELETED, File: /restore/rrl/sub/c.h - deleted: it is not in the incremental saveset
 %VBACKUP-I-RESTSUMM, Files: 10, Bytes: 3108914 - restored
 %VBACKUP-I-COMPLETED, Operation: restore, Seconds: 0.01 - completed
```

   An incremental chain. The full save records its files in the journal of this backup plan. On Monday one file was created and one deleted; `/SINCE=BACKUP` saves the new file and the directories, and the catalog lists the four unchanged files as present. The restore takes the full saveset first, then the incremental one; the file deleted on Monday is deleted from the output. The directories are counted in RESTSUMM each time they are restored.

3.
```
$ vbackup /home /mnt/usb/home.bck /VOLUME_SIZE=4G /DATA_FORMAT=COMPRESSED
 ...
 %VBACKUP-I-CREATED, Volume: /mnt/usb/home.bck.002 - created
 %VBACKUP-I-CREATED, Volume: /mnt/usb/home.bck.003 - created
%VBACKUP-I-SAVESUMM, Files: 48211, Bytes: 10737418240, Blocks: 151216, Volumes: 3 -
saved
 ...
```

```
$ vbackup /mnt/usb/home.bck /restore
```

The saveset is cut into volumes of 4 GB for a FAT32 disk, and its data is compressed. Volumes after the first are reported as they are created. To restore, give the first volume; the others are found beside it. Had a volume been lost, MISSVOL would name it and the files in the other volumes would still be restored.

4.
```
# umask 077; head -c 32 /dev/urandom | base64 > /root/backup.key
# vbackup /home /backup/home.bck /ENCRYPT /KEY_FILE=/root/backup.key
# crontab -l
30 2 * * * VBACKUP_KEY_FILE=/root/backup.key /usr/local/bin/vbackup /home /backup/home-
$(date +\%a).bck /ENCRYPT /REPLACE
```

An encrypted save run by cron. The key file holds the passphrase in its first line and is readable by root only; a key file that others may read is refused with KEYFILE. In the crontab the key file is named by `VBACKUP_KEY_FILE`. Cron has no terminal: without a key file the save would stop with NOKEY instead of waiting. Keep a copy of the key file in a safe place - without it the savesets cannot be opened.

5.
```
$ vbackup /etc backup-host::/backup/etc.bck /VOLUME_SIZE=1M /VERIFY
  %VBACKUP-I-STARTED, Operation: save, Input: /etc, Output: backup-host::/backup/etc.bck -
started
  %VBACKUP-I-STARTED, Operation: copy of a saveset, Input: (standard input), Output: /backup/etc.bck -
started
   %VBACKUP-I-CREATED, Volume: (standard output) - created
   %VBACKUP-I-CREATED, Volume: /backup/etc.bck.002 - created
   ...
   %VBACKUP-I-CREATED, Volume: /backup/etc.bck.005 - created
   %VBACKUP-I-SAVESUMM, Files: 42, Bytes: 3662512, Blocks: 72, Volumes: 5 - saved
   %VBACKUP-I-XFRSUMM, Blocks: 72, Volumes: 5, Bad: 0 - copied
   %VBACKUP-I-COMPLETED, Operation: copy of a saveset, Seconds: 0.02 - completed
   %VBACKUP-I-VERIFYING, Saveset: backup-host::/backup/etc.bck - verifying
  %VBACKUP-I-STARTED, Operation: copy of a saveset, Input: /backup/etc.bck, Output: (standard output)
started
   %VBACKUP-I-XFRSUMM, Blocks: 72, Volumes: 5, Bad: 0 - copied
   %VBACKUP-I-COMPLETED, Operation: copy of a saveset, Seconds: 0.00 - completed
   %VBACKUP-I-CMPSUMM, Files: 42, Differences: 0 - compared
   %VBACKUP-I-COMPLETED, Operation: save, Seconds: 0.04 - completed
```

A save to another node, in volumes of 1 MB, verified. VBACKUP on `backup-host`, started through ssh, receives the stream and makes the volume files `/backup/etc.bck`, `/backup/etc.bck.002`, ... there, checking every block; its messages (the copy of a saveset) come here among those of the save. `/VERIFY` reads the saveset back from there and compares it with `/etc`. Had VBACKUP there failed, REMOTEERR would give its completion code, and the exit status would be 2.

6.
```
$ vbackup backup-host::/backup/etc.bck /restore/etc
  %VBACKUP-I-STARTED, Operation: restore, Input: backup-host::/backup/etc.bck, Output: /restore/etc -
started
  %VBACKUP-I-STARTED, Operation: copy of a saveset, Input: /backup/etc.bck, Output: (standard output)
started
   %VBACKUP-I-XFRSUMM, Blocks: 72, Volumes: 5, Bad: 0 - copied
   %VBACKUP-I-COMPLETED, Operation: copy of a saveset, Seconds: 0.00 - completed
   %VBACKUP-I-RESTSUMM, Files: 42, Bytes: 3662512 - restored
   %VBACKUP-I-COMPLETED, Operation: restore, Seconds: 0.01 - completed
   $ vbackup backup-host::/backup/etc.bck /LIST
   $ vbackup backup-host::/backup/etc.bck /EXTRACT=etc/fstab | less
```

A restore from another node. VBACKUP there sends the volumes back to back; the restore reads them as a pipe, once, forward only. A listing reads the whole stream (the catalog at its end cannot be reached first); an extraction stops as soon as it has its file.

7.
```
$ vbackup /backup/home.bck - > /tmp/home.stream
 %VBACKUP-I-STARTED, Operation: copy of a saveset, Input: /backup/home.bck, Output: (standard output)
started
  %VBACKUP-I-XFRSUMM, Blocks: 72, Volumes: 5, Bad: 0 - copied
  %VBACKUP-I-COMPLETED, Operation: copy of a saveset, Seconds: 0.01 - completed
  $ vbackup - /mnt/usb/home.bck < /tmp/home.stream
 %VBACKUP-I-STARTED, Operation: copy of a saveset, Input: (standard input), Output: /mnt/usb/home.bck
started
  %VBACKUP-I-CREATED, Volume: /mnt/usb/home.bck.002 - created
  ...
  %VBACKUP-I-CREATED, Volume: /mnt/usb/home.bck.005 - created
  %VBACKUP-I-XFRSUMM, Blocks: 72, Volumes: 5, Bad: 0 - copied
  %VBACKUP-I-COMPLETED, Operation: copy of a saveset, Seconds: 0.02 - completed
  $ cmp /backup/home.bck.003 /mnt/usb/home.bck.003
  $ vbackup /backup/secret.bck - | ssh vault 'vbackup - /archive/secret.bck'
  $ vbackup /backup/secret.bck vault::/archive/secret.bck
```

A saveset of five volumes is copied into a stream, here kept in a file, and back into volume files: every volume of the copy is byte for byte the original. The last two commands send an encrypted saveset to another machine, through a pipe given by hand and as *node*::*file*: the blocks are copied, never decrypted, so no passphrase is asked on either side. Had a block gone bad on the way, the receiver would report it (BLKCOPIED) and copy it as it is; a restore repairs it from its group.

8.
```
# umount /dev/sdb1
# vbackup /dev/sdb1 /mnt/usb/sdb1.bck /PHYSICAL /DATA_FORMAT=COMPRESSED
# vbackup /mnt/usb/sdb1.bck /dev/sdc1 /PHYSICAL /REPLACE
Everything on /dev/sdc1 (21474836480 bytes) is to be overwritten with the device saved in /mnt/usb/sd
 Type YES to go on: YES
```

A partition is copied block by block to another one. It is unmounted first (a device mounted read-write is refused with PHYSMOUNTED). The restore requires `/REPLACE` and, at a terminal, `YES`. The copy carries the labels and UUIDs of the original (PHYSUUID): do not mount both at the same time.

9.
```
# vbackup /mnt/data /mnt/usb/data.bck /IMAGE /VERIFY
# vbackup /mnt/usb/data.bck /dev/sdc1 /IMAGE /REPLACE /LOG
```

A whole file system is saved from its mount point, with its type, label and UUID. The restore runs the mkfs program of its type on `/dev/sdc1` (IMGCMD shows the command under `/LOG`), restores the files into the new file system and unmounts it (IMGSUMM). `/dev/sdc1` may be smaller than the original device as long as the files fit.

10. `# vbackup /mnt/usb/home.bck /ORIGINAL %VBACKUP-I-STARTED, Operation: restore, Input: /mnt/usb/home.bck, Output: (where its files came from) - started %VBACKUP-I-ORIGTARGET, Saveset: /mnt/usb/home.bck, Target: /home - its files go back there %VBACKUP-W-FILEEXISTS, File: /home/rrl/a.txt - already exists, not restored ...`

    Every file goes back to the directory it was saved from. Files that are there are kept and reported; add `/REPLACE` to overwrite them.

11. `$ vbackup /home/ivan/old /mnt/usb/old.bck /VERIFY /DELETE ... %VBACKUP-I-CMPSUMM, Files: 213, Differences: 0 - compared %VBACKUP-W-SRCKEPT, File: /home/ivan/old/log.txt - not deleted: it changed after it was saved %VBACKUP-I-DELSUMM, Deleted: 197, Kept: 1`

    The files are archived and then deleted from the disk. A file that was written to after it was saved is kept. Directories are kept.

12. `$ vbackup /backup/rrl.bck /COMPARE $ vbackup /backup/rrl.bck /tmp/restore /COMPARE`

    The first command compares the saveset with the places its files were saved from; the second with a directory into which it was restored. Every difference is reported with COMPARERR; the exit status is 2 when there is one.

13. `$ vbackup /backup/rrl.bck /EXTRACT=rrl/notes.txt | less` `$ vbackup /backup/rrl.bck /tmp/notes.txt /EXTRACT=rrl/notes.txt`

    One file is extracted to the standard output, then to a file. The name is the stored name, as `/LIST` shows it.

14. `$ vbackup /backup/rrl.bck /LIST` `$ vbackup /backup/rrl.bck /LIST /FULL` `$ vbackup /backup/rrl.bck /LIST=/tmp/rrl.lis /FORMAT=LS`

    The three forms of a listing: brief (the default), full, and one line per file in the form of `ls -l`, here written to a file.

15. `$ vbackup /JOURNAL=/backup/home.jnl /LIST /FULL '/SELECT=*/notes.txt'` `$ rm /backup/home.jnl` `$ vbackup /backup/full.bck,/backup/mon.bck /RECORD /JOURNAL=/backup/home.jnl`

    The journal is listed with the files it knows whose names end in `/notes.txt` and the saveset that holds the last copy of each. Then the journal is rebuilt from the catalogs of the savesets, oldest first.

16. `# vbkx t /mnt/usb/home.bck` `/mnt/usb/home.bck: all files read, all checksums match` `# vbkx l /mnt/usb/home.bck | less` `# vbkx x /mnt/usb/home.bck -C /mnt/root/home rrl/projects`

    On a rescue system without VBACKUP, the statically linked vbkx tests the saveset, lists it and extracts one directory into the disk being repaired.

17. `C:\> set VBACKUP_KEY_FILE=C:\Users\rrl\backup.key` `C:\> vbkx.exe l E:\home.bck` `C:\> vbkx.exe x E:\home.bck -C C:\restore rrl/documents`

    On Windows, vbkx.exe lists an encrypted saveset on a USB disk and extracts a directory. Names that Windows cannot hold are not extracted, and said.

18. `$ vbackup USERS.BCK /LIST /FULL` `$ vbackup USERS.BCK /restore/vms /SELECT=SMITH/*` `$ vbkx p USERS.BCK "[SMITH]LOGIN.COM;3"`

    A saveset of OpenVMS BACKUP, copied in binary mode from an Alpha, is listed as `BACKUP/LIST/FULL` lists it, the files of one user restored with Linux names, and one older version of a file displayed by vbkx.

Chapter 5. Command Dictionary

This chapter describes the `vbackup` command: its format and parameters, how the operation is determined, its exit status and restrictions (Section 5.1), and each of its qualifiers in alphabetical order (Section 5.2).

5.1 The VBACKUP Command

The VBACKUP utility saves files into savesets, restores, lists, compares, extracts and copies them, here or on another node, copies files from disk to disk, copies block devices and whole file systems, and keeps the journal of incremental backup.

Format

```
vbackup input-specifier[,...] output-specifier [/qualifiers]
vbackup input-specifier /LIST[=file] [/qualifiers]
vbackup input-specifier [output-specifier] /COMPARE [/qualifiers]
vbackup input-specifier [output-specifier] /EXTRACT=stored-name [/qualifiers]
vbackup input-specifier /ORIGINAL [/qualifiers]
vbackup input-saveset output-saveset [/TRANSFER] [/qualifiers]
vbackup saveset[,...] /RECORD [/JOURNAL=file]
vbackup /JOURNAL[=file] /LIST [/FULL] [/SELECT=(pattern[,...])]
vbackup /HELP [topic ...]
```

Parameters

input-specifier[,...]

Specifies the input of the operation, as one word; several specifications are separated by commas, without spaces. An element in double quotes is taken as it stands.

- **Save, copy:** the files and directories to be saved or copied, with optional wildcards (see Section 3.4). A directory is taken with everything in it. At most 64 specifications.
- **Save /PHYSICAL:** one block device or image file.
- **Save /IMAGE:** one mount point, or one mounted block device.
- **Restore, list, compare, extract, journal rebuild:** the saveset, given by the name of its first volume; the other volumes are found beside it. For a restore and a journal rebuild several savesets may be given; they are processed one after the other. For a restore `/INCREMENTAL`, give the full saveset first and then the incremental ones in the order they were made.
- **Copy of a saveset:** one saveset, given by the name of its first volume.
- **`-`:** a saveset read from the standard input (restore, `/LIST`, `/COMPARE`, `/EXTRACT`, copy of a saveset).
- **`node::file`:** a saveset on another node (Section 3.10), read through a pipe as `-` is; it must be the only input. *node* is a host name or `user@host`, without a slash.

A saveset is recognized by its contents. An input that contains a wildcard or `...` is never a saveset.

output-specifier

Specifies the output of the operation.

- **Save:** the saveset to be created: a name ending in `.bck` or `.sav`, any name with `/SAVE_SET`, `-` for the standard output, or *node*`::`*file* for a saveset on another node. If the saveset exists, VBACKUP stops (OPENOUT, errno 17) unless `/REPLACE` is given.
- **Copy of a saveset:** the saveset to be created, named as for a save, or any name with `/TRANSFER`. If a volume of it exists, VBACKUP stops (OPENOUT, errno 17) unless `/REPLACE` is given.
- **Restore:** the directory to restore into; it is created if necessary. Trailing slashes are ignored. Omitted with `/ORIGINAL`.
- **Restore /PHYSICAL:** the output block device, or an image file.
- **Restore /IMAGE:** the output block device.
- **Compare:** the directory to compare with. When omitted, every file is compared with the place it was saved from (the bases of the saveset).

- **Extract:** the file to be written. When omitted, or `-`, the file goes to the standard output.
- **Copy:** the directory to copy into. An output that is not a saveset name, beside an input that is not a saveset, means a copy.
- **List, journal rebuild, journal listing:** none.

Only these two parameters are taken. A third word is an error (MAXPARM); most often it is a qualifier typed without its slash.

Usage Summary

VBACKUP is invoked from the shell. Qualifiers may be placed anywhere on the command line, before, between or after the parameters. A qualifier may be abbreviated as long as it stays unique (`/VER` for `/VERIFY`); an ambiguous abbreviation is an error. A qualifier may also be written in the Unix way, `--VERIFY`; the case of letters does not matter. A word that begins with a slash is a qualifier only when it names one, so `/etc` and `/home` are file names. A lone `--` ends the qualifiers: every word after it is a parameter. `-h` is the same as `/HELP`.

Qualifiers may be glued to a parameter, as in DCL: `x.sav/sav/log` is `x.sav /SAVE_SET /LOG`. VBACKUP reports it (GLUED), so that a mistyped file name is not taken for a qualifier unnoticed. A word that is the name of an existing file is never cut; a qualifier whose value contains a slash (`/JOURNAL=/var/...`) must be given apart.

Wildcards are expanded by VBACKUP itself; quote them, and quote qualifier values that contain parentheses or `*`:

```
$ vbackup '/home/rrl/.../*.c' sources.bck '/EXCLUDE=(*.o,*/.cache)'
```

How the Operation Is Determined

VBACKUP examines the parameters in this order:

1. Each input is probed. An input with a wildcard is not a saveset; any other input is a saveset when its contents say so. A *node*::*file* stands for `-` on its side: as the input it is a saveset, as the output a saveset name.
2. `/EXTRACT` given: extract. `/COMPARE` given: compare. Both require exactly one input, and it must be a saveset (otherwise NOTSAVESET).
3. One input that is a saveset, an output that is a saveset name (`.bck`, `.sav`, `/SAVE_SET` or `-`) or `/TRANSFER` given, and no `/LIST`: copy of a saveset, block for block. So `vbackup x.bck y.bck` copies the saveset; to restore, give a directory whose name is not a saveset name.
4. Every input is a saveset: `/LIST` without an output - list; `/RECORD` without an output - journal rebuild; otherwise - restore.
5. The output is a saveset name (`.bck`, `.sav`, `/SAVE_SET` or `-`): save.
6. An output is given and the inputs exist: copy.
7. No parameter at all, with `/JOURNAL` and `/LIST`: journal listing.

`/TRANSFER` with any other outcome is refused (QUALUSE).

Otherwise VBACKUP reports what is missing: OPENIN when the input does not exist and no output is given; NOPARAM when the output is missing; IVOP when an input does not exist and the output is not a saveset name. With no parameters and no qualifiers, VBACKUP displays its usage text.

Started and Completed

Every operation except a listing reports its beginning (STARTED) and its end (COMPLETED) with the elapsed time and the outcome: `completed`, `completed with warnings` or `completed with errors`. Save, restore, compare, copy and copy of a saveset also report their totals (SAVESUMM, RESTSUMM, CMPSUMM, CPYSUMM, XFRSUMM) without `/LOG`. `/LOG` adds a message for every file.

Messages

All messages are written to the standard error, one per line, in the form

```
dd-mm-yyyy hh:mm:ss.cc pid %VBACKUP-s-IDENT, text
```

The standard output carries only what the operation produces: a listing, the data of /EXTRACT, a saveset written to -, the help text. See Chapter 6. The messages of VBACKUP on another node (Section 3.10) come to the standard error too, with the process number of that side.

Exit Status

Value	Meaning
0	All was done. No warning or error was signalled.
1	All was done, with warnings: at least one message of severity W (for example, a file changed while it was saved, a file exists and was not restored).
2	Something was not done: at least one message of severity E or F was signalled (for example, a file could not be read, a difference was found by /COMPARE or /VERIFY, VBACKUP on another node did not complete - REMOTEERR), or the command could not be parsed.

Informational and success messages do not affect the exit status.

Restrictions

Privileges

- Root privilege is needed to restore files to their original owners (/OWNER=ORIGINAL is the default for root only), to restore device files, to set some extended attributes (trusted.*, security.*, file capabilities) and some chattr flags (immutable, append-only), and to read files the user cannot read.
- /PHYSICAL and /IMAGE operations on block devices require root: they open devices, and a restore /IMAGE runs mkfs and mounts the new file system.
- A user other than root restores files as his own (/OWNER=DEFAULT); owners, and attributes that cannot be set, are reported (ATTRERR).

Limits

Item	Limit
Block size (/BLOCK_SIZE)	8192 to 1048576 bytes, a multiple of 512
Group size (/GROUP_SIZE)	0 to 100 data blocks
Volume size (/VOLUME_SIZE)	at least (group size + 3) x block size; rounded down to a multiple of the block size
Volumes of a saveset read	up to 9999; the search stops after 16 missing names in a row
Input specifications	64
Patterns of /SELECT, of /EXCLUDE	64 each
Depth of directories walked	256 levels
File specification	4095 bytes
Text of /COMMENT	255 bytes

Item	Limit
Command line kept in the summary	4096 bytes
Passphrase	1 to 1024 bytes
PBKDF2 iterations	at least 1000 (default 600000)
Data in one DATA or DATAZ record	1048576 bytes

Other Restrictions

- A saveset written to the standard output carries its volumes back to back; it cannot be verified (hence not deleted after) nor listed by the command that writes it.
- A saveset read from the standard input is read once, forward only: no catalog lookup; vbkx refuses names for it.
- A saveset made on another node takes neither `/DELETE` nor `/LIST`; `/VERIFY` reads it back from there. A *node*::*file* input must be the only input. The other node needs VBACKUP X01-11 or later and must let this one in by its ssh keys.
- A copy of a saveset keeps the block size, group size and volumes of the input: `/BLOCK_SIZE`, `/GROUP_SIZE` and `/VOLUME_SIZE` do not change them.
- `/PHYSICAL` and `/IMAGE` take one input on a save and one saveset on a restore.
- `/INCREMENTAL` refuses a saveset without a catalog and a saveset made before X01-02 (NOTINCR).
- `/ORIGINAL` requires a saveset made by X01-02 or later (ORIGNOBASE); with `/INCREMENTAL` it requires a saveset of one base.
- `/IMAGE` restores only file systems of the types ext2, ext3, ext4, xfs, btrfs, vfat and msdos.
- Savesets are never modified in place; nothing can be added to or deleted from an existing saveset.

5.2 Qualifiers

This section describes the qualifiers of the VBACKUP command in alphabetical order, and the environment variables that affect it.

Qualifier Summary

In the column *Applies to*: S - save, R - restore, L - list, C – compare, X - extract, P - copy, J - journal rebuild and listing, T – copy of a saveset.

Qualifier	Class	Applies to	Default
`/BEFORE=time`	Input file-selection	S, P	All times
`/BLOCK_SIZE=n`	Output save-set	S	65536
`/BRIEF`	Listing	L	`/BRIEF`
`/BY_OWNER=user`	Input file-selection	S, P	All owners
`/CHANGED`	Input file-selection	S, P	`/MODIFIED`
`/COMMENT="text"`	Output save-set	S	None
`/COMPARE`	Command	C	-
`/[NO]CONFIRM`	Command	S, R, P	`/NOCONFIRM`
`/CREATED`	Input file-selection	S, P	`/MODIFIED`
`/[NO]CROSS_DEVICE`	Input file-selection	S, P	`/NOCROSS_DEVICE`
`/DATA_FORMAT=keyword`	Output save-set	S	`UNCOMPRESSED`
`/DELETE`	Input file	S	Files are kept
`/ENCRYPT`	Output save-set	S	Not encrypted
`/EXCLUDE=(pattern[,...])`	Input file-selection	S, R, C, P	None
`/EXTRACT=stored-name`	Command	X	-
`/FORMAT=keyword`	Listing	L	`VMS`
`/FULL`	Listing	L, J	`/BRIEF`
`/GROUP_SIZE=n`	Output save-set	S	10
`/HELP [topic ...]`	Command	-	-
`/IGNORE=NOBACKUP`	Input file-selection	S, P	nodump files left out
`/IMAGE`	Command	S, R	-
`/INCREMENTAL`	Output file	R	-
`/JOURNAL[=file]`	Command	S, J	See text
`/KEY_FILE=file`	Command	S, R, L, C, X, J	`VBACKUP_KEY_FILE`, else the terminal
`/LEVEL=n`	Output save-set	S	1 (with `/DATA_FORMAT=COMPRESSED`)
`/LIST[=file]`	Command	L, S	-
`/[NO]LOG`	Command	all	`/NOLOG`
`/MODIFIED`	Input file-selection	S, P	`/MODIFIED`
`/ORIGINAL`	Output file	R	-
`/OWNER=option`	Output file	R, P	`ORIGINAL` for root, `DEFAULT` otherwise
`/PARITY=m`	Output save-set	S	1
`/PHYSICAL`	Command	S, R	-
`/[NO]RECORD`	Command	S, J	`/NORECORD`
`/[NO]REPLACE`	Output file, Output save-set	S, R, X, T	`/NOREPLACE`
`/SAVE_SET`	Output save-set	S, T	By name
`/SELECT=(pattern[,...])`	Input file-selection	S, R, C, P, J	All files
`/SINCE=time`	Input file-selection	S, P	All times
`/[NO]SOLID`	Output save-set	S	`/SOLID` when compressing
`/TRANSFER`	Command	T	By name
`/[NO]VERIFY`	Command	S, P	`/NOVERIFY`
`/VOLUME_SIZE=size`	Output save-set	S	One volume

Qualifier	Class	Applies to	Default
/[NO]XATTRS	Input file, Output file	S, R, P	/XATTRS

/BEFORE

Input file-selection qualifier.

Format

```
/BEFORE=time
```

Description

Selects for saving only the files whose time is earlier than the time given. The time examined is the modification time, unless /CREATED or /CHANGED is given. Directories are always saved.

The time is written as follows (an unambiguous abbreviation of a keyword is accepted):

Form	Meaning
dd-MMM-yyyy[hh:mm[:ss]]	An absolute time, for example 3-OCT-2026 14:00
hh:mm[:ss]	Today at that time
-[dd]hh:mm[:ss]	A delta time back from now: "-1 0:0" is one day ago
TODAY, YESTERDAY, TOMORROW	Midnight at the beginning of that day
NOW	The present moment

/BEFORE makes the saveset incremental: the files it does not choose are listed in the catalog as PRESENT (see Section 3.5). An ill-formed time is reported with IVTIME. /BEFORE cannot be combined with /PHYSICAL or /DELETE. In a copy, only the chosen files are copied.

Example

```
$ vbackup /home/rrl old.bck "/BEFORE=1-JAN-2026"
```

Saves the files of /home/rrl last modified before 1 January 2026; the catalog lists the newer files as present.

/BLOCK_SIZE

Output save-set qualifier.

Format

```
/BLOCK_SIZE=n
```

Description

Specifies the size, in bytes, of the blocks of the saveset. *n* must be a multiple of 512 from 8192 to 1048576; the default is 65536. Larger blocks are slightly faster; smaller blocks lose less data when one is lost. The block size of a saveset being read is taken from the saveset.

An illegal value is reported with IVQUAL.

Example

```
$ vbackup /home home.bck /BLOCK_SIZE=1000
%VBACKUP-E-IVQUAL, Value: 1000, Qualifier: /BLOCK_SIZE - is not valid
$ vbackup /home home.bck /BLOCK_SIZE=262144
```

/BRIEF

Listing qualifier.

Format

`/BRIEF`

Description

With `/LIST`, displays for every file its stored name, its size and its modification date; directories end with a slash. This is the default. `/BRIEF` and `/FULL` cannot be given together (CONFQUAL).

Example

```
$ vbackup rrl.bck /LIST /BRIEF
Listing of save set(s)

Save set:          rrl.bck
Volumes:           1
Written with:      VBACKUP X01-11
Node name:         TTR-RTR
Written by:        root
Command:           vbackup /home/rrl rrl.bck /VERIFY
Date:               5-OCT-2026 14:19:25.00
Base:              /home
Kind:              full
Block size:        65536
Group size:        10
Operating system:  Linux 6.1.0-52-amd64 x86_64

rrl/
rrl/a.txt                                              6   5-OCT-2026 14:19
rrl/link                                               5   5-OCT-2026 14:19
rrl/sub/
rrl/sub/b.c                                       108894   5-OCT-2026 14:19
rrl/sub/c.h                                            2   5-OCT-2026 14:19

Total of 6 files, 108902 bytes
End of save set
```

/BY_OWNER

Input file-selection qualifier.

Format

`/BY_OWNER=user`

Description

Selects only the files owned by the user given, by name or by numeric user ID. Directories are always taken. Files of other owners are not covered: they are neither saved nor listed as present. An unknown name that is not a number is reported with IVQUAL. `/BY_OWNER` cannot be combined with `/PHYSICAL` or `/IMAGE`.

Example

```
# vbackup /home ivan.bck /BY_OWNER=ivan
```

/CHANGED

Input file-selection qualifier.

Format

```
/CHANGED
```

Description

Makes `/SINCE` and `/BEFORE` examine the change time (ctime) of a file. The change time moves also when the permissions, the owner or the name of a file change. Only one of `/MODIFIED`, `/CREATED` and `/CHANGED` may be given (CONFQUAL).

Example

```
$ vbackup /etc etc.bck /SINCE=YESTERDAY /CHANGED
```

/COMMENT

Output save-set qualifier.

Format

```
/COMMENT="text"
```

Description

Stores the text, at most 255 bytes, in the summary of the saveset. `/LIST` displays it in the line `Comment:`.

Example

```
$ vbackup /home/rrl rrl.bck /COMMENT="before the upgrade"
```

/COMPARE

Command qualifier.

Format

```
input-specifier [output-specifier] /COMPARE
```

Description

Reads the saveset and compares every file with the disk: the type, the size, the contents, and the target of a symbolic link. Owners, modes and times are not compared. With an output specifier, the files are looked for under that directory by their stored names; without it, under the bases the saveset recorded (the places they were saved from). The first difference of each file is reported with COMPARERR; the totals with CMPSUMM. Any difference makes the exit status 2.

A file of the catalog that the record stream did not give is a difference too (since X01-22): `not compared: its records were lost in bad blocks` after lost blocks, else `not compared: the catalog has it, the record stream does not`. A file missing from the saveset is never passed over in silence.

`/SELECT` and `/EXCLUDE` restrict the comparison. The saveset may be `-`. The input must be exactly one saveset (otherwise NOTSAVESET).

Example

```
$ vbackup rrl.bck /COMPARE
%VBACKUP-I-STARTED, Operation: compare, Input: rrl.bck - started
%VBACKUP-E-COMPARERR, File: /home/rrl/a.txt - the size differs
%VBACKUP-I-CMPSUMM, Files: 6, Differences: 1 - compared
%VBACKUP-I-COMPLETED, Operation: compare, Seconds: 0.00 - completed with errors
```

/CONFIRM

Command qualifier.

Format

`/CONFIRM` `/NOCONFIRM` (default)

Description

Asks at the terminal before every file is processed:

Operation	Question
Save	`Save` *stored-name* `? [N]:`
Restore	`Restore` *file* `? [N]:`
Copy	`Copy` *file* `? [N]:`
`/DELETE`, deletions of `/INCREMENTAL`	`Delete` *file* `? [N]:`

Answer `YES`, `NO`, `QUIT` or `ALL`; the first letter is enough, and an empty answer is `NO`. `ALL` processes the rest without asking. `QUIT` stops the operation; a save stopped so still closes its saveset properly. When there is no terminal, the answer is `QUIT`. `/CONFIRM` is refused on a restore `/IMAGE` (CONFQUAL).

Example

```
$ vbackup /home/rrl rrl.bck /CONFIRM
Save rrl/a.txt ? [N]: Y
Save rrl/core ? [N]: N
Save rrl/sub/b.c ? [N]: ALL
```

/CREATED

Input file-selection qualifier.

Format

`/CREATED`

Description

Makes `/SINCE` and `/BEFORE` examine the creation (birth) time of a file. Linux keeps it on ext4, xfs and btrfs. A file system without it has no creation time, and its files are not chosen by `/SINCE` or `/BEFORE`. Only one of `/MODIFIED`, `/CREATED` and `/CHANGED` may be given.

Example

```
$ vbackup /data new.bck /SINCE=TODAY /CREATED
```

/CROSS_DEVICE

Input file-selection qualifier.

Format

`/CROSS_DEVICE` `/NOCROSS_DEVICE` (default)

Description

By default a mount point met during the walk is saved as a directory, but what is mounted on it is not; saving `/` therefore does not save `/proc`, `/sys` or a mounted USB disk. `/CROSS_DEVICE` descends into the other file systems too. A save `/IMAGE` always works as `/NOCROSS_DEVICE`.

Example

```
# vbackup / root.bck /NOCROSS_DEVICE '/EXCLUDE=(tmp,var/tmp)'
```

/DATA_FORMAT

Output save-set qualifier.

Format

`/DATA_FORMAT=COMPRESSED` `/DATA_FORMAT=UNCOMPRESSED` (default)

Description

`COMPRESSED` stores the data of the files compressed in the LZ4 block format (DATAZ records); data that does not shrink is stored as it is. The summary records the codec, and `/LIST` displays `Data format: compressed (LZ4)` - or Deflate, LZMA with `/LEVEL`. The compression runs on several threads (`VBACKUP_ZTHREADS`); the saveset does not depend on their number. Readers recognize compressed data by themselves. `/DATA_FORMAT` works also with `/PHYSICAL` and `/IMAGE`. The small files are compressed in groups, as SOLID records, unless `/NOSOLID` is given (see `/SOLID`). See Section 3.7.

Example

```
$ vbackup /home /mnt/usb/home.bck /DATA_FORMAT=COMPRESSED
```

/DELETE

Input file qualifier.

Format

`/DELETE`

Description

After a save, deletes from the disk the files that were saved and verified. `/VERIFY` is required (otherwise QUALUSE). A file is deleted only when the verification found no difference at all in the whole saveset, and the file has not changed since it was saved: the same inode and device, and - for other than a further name of a hard link - the same size, modification time and change time as its catalog entry. Otherwise the file is kept and SRCKEPT tells why. Only regular files, symbolic links and hard links with the status OK are deleted; directories are kept.

`/CONFIRM` asks before each deletion; `/LOG` reports each one (SRCDELETED); DELSUMM gives the totals. `/DELETE` cannot be combined with `/PHYSICAL`, `/IMAGE`, `/SINCE` or `/BEFORE` (CONFQUAL). With the standard output as the saveset it is refused (QUALUSE): there is nothing to verify against. With a saveset on another node it is refused too (QUALUSE: `not with a saveset made on another node`).

Example

```
$ vbackup /home/ivan/old /mnt/usb/old.bck /VERIFY /DELETE /LOG
...
%VBACKUP-I-CMPSUMM, Files: 3, Differences: 0 - compared
%VBACKUP-I-SRCDELETED, File: /home/ivan/old/x - deleted: it is in the saveset and verified
%VBACKUP-I-SRCDELETED, File: /home/ivan/old/y - deleted: it is in the saveset and verified
%VBACKUP-I-DELSUMM, Deleted: 2, Kept: 0
```

/ENCRYPT

Output save-set qualifier.

Format

`/ENCRYPT`

Description

Makes an encrypted saveset (see Section 3.8). The passphrase is read from the key file of `/KEY_FILE` or of `VBACKUP_KEY_FILE`; otherwise it is asked twice at the terminal, without echo.

Without a key file and a terminal the save stops with NOKEY; when the two answers differ, with KEYMATCH. With /LOG, ENCRYPTED reports the iteration count.

/ENCRYPT is given to a save only. An encrypted saveset is recognized by itself when it is read: a restore, /LIST, /COMPARE, /EXTRACT and a journal rebuild ask for the passphrase or take /KEY_FILE. /ENCRYPT given to another operation is refused (QUALUSE).

Example

```
$ vbackup /home/me me.bck /ENCRYPT
Passphrase for me.bck:
The same passphrase again:
```

/EXCLUDE

Input file-selection qualifier.

Format

```
/EXCLUDE=(pattern[,...])
```

Description

Leaves out the files whose stored name matches one of the patterns. A directory that matches is left out with everything in it. In a pattern, * matches any characters including /, and % and ? match one character. At most 64 patterns (TOOMANY).

On a save and a copy, the excluded files are not covered; on a restore and a comparison they are skipped. A saveset listing leaves the excluded entries out. It cannot be combined with /INCREMENTAL, /PHYSICAL or /IMAGE.

Example

```
$ vbackup /home/rrl rrl.bck '/EXCLUDE=(*.o,*/.cache)'
```

/EXTRACT

Command qualifier.

Format

```
input-specifier [output-specifier] /EXTRACT=stored-name
```

Description

Writes the contents of one regular file of the saveset to the output specifier, or to the standard output when it is omitted or -. The name is exact, without wildcards, as /LIST shows it. The file is reached through the catalog, so this is fast even in a very large saveset; from a pipe or a saveset without a catalog it is found by reading. The data is checked against the file CRC (CRCERR).

A name that is not in the saveset gives NOTFOUND; a file that is not regular gives UNSUPP. An existing output file is kept (FILEEXISTS) unless /REPLACE is given.

Example

```
$ vbackup home.bck /EXTRACT=rrl/notes.txt | less
$ vbackup home.bck /tmp/notes.txt /EXTRACT=rrl/notes.txt
```

/FORMAT

Listing qualifier.

Format

/FORMAT=VMS (default) /FORMAT=LS

Description

`VMS` is the listing described under `/BRIEF` and `/FULL`. `LS` prints, with no summary, one line per file like `ls -l` - mode, link count, owner, group, size, date and time (`mm-dd-yyyy hh:mm:ss`), the full stored name and, for a symbolic link, `-> target`. It is meant for programs; the Midnight Commander plugin reads it.

Example

```
$ vbackup rrl.bck /LIST /FORMAT=LS
drwxr-xr-x   3 root     root           4096 10-05-2026 14:19:25 rrl
-rw-r--r--   1 root     root              6 10-05-2026 14:19:25 rrl/a.txt
lrwxrwxrwx   1 root     root              5 10-05-2026 14:19:25 rrl/link -> a.txt
drwxr-xr-x   2 root     root           4096 10-05-2026 14:19:25 rrl/sub
-rw-r--r--   1 root     root         108894 10-05-2026 14:19:25 rrl/sub/b.c
-rw-r--r--   1 root     root              2 10-05-2026 14:19:25 rrl/sub/c.h
```

/FULL

Listing qualifier.

Format

```
/FULL
```

Description

With `/LIST`, displays for every file its type, size, owner (name and number), protection, modification time, checksum and status, and the target of a link. The types are `file`, `directory`, `symlink`, `hard link`, `character device`, `block device`, `FIFO` and `socket`; the statuses `OK`, `changed while saved` and `read error`.

With `/JOURNAL /LIST`, adds the files the journal knows, each with its size, the time it was recorded and the saveset that holds its last copy.

Example

```
$ vbackup rrl.bck /LIST /FULL
...
rrl/a.txt
    Type: file         Size: 6             Owner: root:root (0,0)  Protection: -rw-r--r--
    Modified:  5-OCT-2026 14:19:25.00  Checksum: 363A3020  Status: OK
rrl/link
    Type: symlink      Size: 5             Owner: root:root (0,0)  Protection: lrwxrwxrwx
    Modified:  5-OCT-2026 14:19:25.00  Checksum: 00000000  Status: OK
    Link to: a.txt
...
```

/GROUP_SIZE

Output save-set qualifier.

Format

```
/GROUP_SIZE=n
```

Description

Specifies the number of data blocks after which an XOR block is written. *n* is 0 to 100; the default is 10, which makes the saveset about 10% larger. One bad block in a group is rebuilt when the saveset is read (BLKFIXED). A smaller *n* repairs more damage and costs more space. `/GROUP_SIZE=0` writes no XOR blocks, and nothing can be repaired.

Example

```
$ vbackup /home /mnt/usb/home.bck /GROUP_SIZE=4
```

/HELP

Command qualifier.

Format

```
/HELP [topic ...]
```

Description

Displays the description of VBACKUP from its help library, or of the topic given. Every word after `/HELP` is part of the topic, a qualifier too: `vbackup /HELP /VERIFY`. Topics include the qualifiers, PARAMETERS, WILDCARDS, INCREMENTAL, TIME, PLUGINS, VBKX, EXAMPLES, TROUBLESHOOTING and MESSAGES. A topic that does not exist is reported with NOTOPIC. `-h` is the same as `/HELP`. When VBACKUP was built without the HELP package, the usage text is displayed instead.

The library is looked for in the file named by `VBACKUP_HELPLIB`, then in `../share/help/vbackup.hlb` relative to the image, beside the image, and where the product was installed.

Example

```
$ vbackup /HELP /VERIFY

  VBACKUP /VERIFY

    After the saveset is written, VBACKUP reads it again and compares
    every file with the disk. A difference is reported with
    %VBACKUP-E-COMPARERR.
```

/IGNORE

Input file-selection qualifier.

Format

```
/IGNORE=NOBACKUP
```

Description

Files and directories that carry the nodump flag (`chattr +d`) are normally not saved; this is the NOBACKUP flag of OpenVMS. With `/LOG` each one is reported (SKIPPED). `/IGNORE=NOBACKUP` saves them too. A save `/IMAGE` always ignores the flag.

Example

```
$ vbackup /home/rrl rrl.bck /IGNORE=NOBACKUP
```

/IMAGE

Command qualifier.

Format

```
mount-point|device saveset /IMAGE saveset device /IMAGE /REPLACE
```

Description

On a save, saves every file of one whole file system and its identity (see Section 3.6). The input is one mount point, or one mounted block device; a subdirectory is not a volume (IMGNOTVOL), an unmounted device is refused (IMGNOTMNT). When the UUID of the file system cannot be found, IMGNOID is reported and the new file system will get a new UUID.

On a restore, makes a new file system of the saved type, label and UUID on the output device and restores the files into it. It requires `/REPLACE` and refuses a mounted or busy

device (PHYSMOUNTED, PHYSHELD), a device that cannot hold the files (IMGSMALL), and a type VBACKUP does not make (IMGUNSUPP). At a terminal VBACKUP asks:

```
Everything on device (n bytes) is to be overwritten with the device saved in saveset.
Type YES to go on:
```

Anything but `YES` leaves the device untouched (PHYSABORT). Without a terminal on the standard input nothing is asked. `/LOG` reports the mkfs command (IMGCMD). The totals are reported by IMGSUMM.

`/IMAGE` works with `/RECORD`, `/SINCE`, `/VERIFY`, `/DATA_FORMAT` and `/ENCRYPT`; it refuses `/SELECT`, `/EXCLUDE`, `/BY_OWNER`, `/INCREMENTAL`, `/PHYSICAL`, `/ORIGINAL`, `/DELETE` and, on a restore, `/CONFIRM`. A save `/IMAGE` takes one input, a restore one saveset (QUALUSE).

Example

```
# vbackup /mnt/data /mnt/usb/data.bck /IMAGE
# vbackup /mnt/usb/data.bck /dev/sdc1 /IMAGE /REPLACE
```

/INCREMENTAL

Output file qualifier.

Format

```
full-saveset,incremental-saveset[,...] directory /INCREMENTAL
```

Description

Restores a chain of savesets: the full saveset, then every incremental one, oldest first, and makes every directory of the savesets look as it did at the last save. Before the files of each saveset are touched, its catalog is read; after the saveset is restored, a file that is in a directory of the saveset but is not listed in its catalog is deleted (DELETED). A file the catalog lists as present, but that no earlier saveset brought, is reported (MISSING). `/INCREMENTAL` implies `/REPLACE`.

Only directories that are in the saveset are cleaned, never the output directory itself. A saveset without a catalog, with a damaged catalog, or made before X01-02 is refused (NOTINCR) and ends the chain: nothing is deleted on the word of an incomplete list. `/CONFIRM` asks before every deletion, `/LOG` reports it. `/SELECT` and `/EXCLUDE` cannot be given with `/INCREMENTAL` (CONFQUAL), nor can `/PHYSICAL` or `/IMAGE`. With `/ORIGINAL` the saveset must have one base.

Example

```
$ vbackup full.bck,mon.bck,tue.bck /home /INCREMENTAL
```

/JOURNAL

Command qualifier.

Format

```
/JOURNAL[=file]
```

Description

Names the journal used by `/RECORD` and `/SINCE=BACKUP`. The default is `/var/lib/vbackup/vbackup.jnl` for root and `$HOME/.vbackup/vbackup.jnl` for other users; the directory is created when needed. One journal for each backup plan is a common use.

With `/LIST` and no parameter, the journal is listed: the savesets it knows, each with its time, kind, number of files and bytes, its name and filter. `/FULL` adds the files; `/SELECT` picks files by their absolute names. A journal that cannot be read or written is reported with JNLERR.

Example

```
$ vbackup /JOURNAL=/backup/plan1.jnl /LIST
Journal:           /backup/plan1.jnl

 5-OCT-2026 14:20:37.00  full              7 files        3108910 bytes  /backup/full.bck
 5-OCT-2026 14:20:38.00  incremental     3 files          4 bytes  /backup/mon.bck /SINCE=BACKUP

Total of 2 savesets, 6 files recorded
```

/KEY_FILE

Command qualifier.

Format

`/KEY_FILE=file`

Description

Names the key file that holds the passphrase of an encrypted saveset: its first line, without the line end. The file must be a regular file that only its owner may read or write (`chmod 600`); otherwise it is refused (KEYFILE). Without `/KEY_FILE`, the file named by `VBACKUP_KEY_FILE` is used; without both, the passphrase is asked at the terminal. `/KEY_FILE` serves a save `/ENCRYPT` and every reading of an encrypted saveset; for a saveset that is not encrypted it has no effect.

Example

```
$ chmod 600 /root/backup.key
# vbackup /home /backup/home.bck /ENCRYPT /KEY_FILE=/root/backup.key
```

/LEVEL

Output save-set qualifier.

Format

`/LEVEL=n` (1 .. 9; default 1 with `/DATA_FORMAT=COMPRESSED`)

Description

How hard the data is compressed: 1 LZ4, 2 to 5 Deflate, 6 to 9 LZMA, at growing effort (Section 3.7). It compresses by itself - `/DATA_FORMAT= COMPRESSED` is implied; with `/DATA_FORMAT=UNCOMPRESSED` it is refused (CONFQUAL), a value out of 1 .. 9 too (IVQUAL). `/LIST` displays the codec, `Data format: compressed (LZ4)`, `(Deflate)` or `(LZMA)`. Every record is checked after it is compressed (ZCHECK). Works with `/PHYSICAL`, `/IMAGE`, `/ENCRYPT`, `/PARITY`. The small files go into SOLID records, unless `/NOSOLID` (see `/SOLID`).

Example

```
$ vbackup /home /mnt/usb/home.bck /LEVEL=6
```

/LIST

Command qualifier.

Format

`/LIST[=file]`

Description

Displays the summary of the saveset - the name, the number of volumes, the product, node, user, command, date, bases, kind and filter, block, group and volume sizes, data format, comment, encryption and, for `/PHYSICAL` and `/IMAGE` savesets, the device or file system - and then its files, in the form chosen by `/BRIEF`, `/FULL` or `/FORMAT`. The files are read from the catalog; without a catalog (NOTRAILER) the whole saveset is read (NOCATALOG). The

present entries of an incremental saveset are not listed. With a file name, the listing is written into that file, which is overwritten.

Given to a save, /LIST lists the new saveset (refused when it goes to the standard output or to another node). A saveset on another node, *node*::*file*, is listed from the stream it sends (Section 3.10). With /JOURNAL and no parameter, it lists the journal. /SELECT and /EXCLUDE restrict a saveset listing as they restrict a restore.

Example

```
$ vbackup /backup/rrl.bck /LIST=/tmp/rrl.lis /FULL
```

/LOG

Command qualifier.

Format

/LOG /NOLOG (default)

Description

Reports every file saved (SAVED), restored (RESTORED), compared (COMPARED), copied (COPIED) or deleted (SRCDELETED, DELETED), every volume created (CREATED; without /LOG only volumes 2 and later), files skipped for the nodump flag (SKIPPED), the encryption (ENCRYPTED) and the mkfs command (IMGCMD). Without /LOG VBACKUP reports the beginning, the totals and the end of the operation.

Example

```
$ vbackup rrl.bck /tmp/out /LOG
%VBACKUP-I-STARTED, Operation: restore, Input: rrl.bck, Output: /tmp/out - started
%VBACKUP-I-RESTORED, File: /tmp/out/rrl - restored
%VBACKUP-I-RESTORED, File: /tmp/out/rrl/a.txt - restored
...
%VBACKUP-I-RESTSUMM, Files: 6, Bytes: 108902 - restored
%VBACKUP-I-COMPLETED, Operation: restore, Seconds: 0.00 - completed
```

/MODIFIED

Input file-selection qualifier.

Format

/MODIFIED

Description

Makes /SINCE and /BEFORE examine the modification time (mtime) of a file. This is the default. Only one of /MODIFIED, /CREATED and /CHANGED may be given.

Example

```
$ vbackup /home home.bck /SINCE=TODAY /MODIFIED
```

/ORIGINAL

Output file qualifier.

Format

saveset /ORIGINAL

Description

Restores every file into the directory it was saved from; no output specifier is given (otherwise QUALUSE). The places are the absolute bases recorded in the summary since X01-02;

a saveset made before does not know them (ORIGNOBASE). Before writing, VBACKUP reports where the files go (ORIGTARGET). A saveset of several input specifications puts each file back under its own base. Files that exist are kept unless `/REPLACE` is given. `/ORIGINAL` cannot be combined with `/PHYSICAL` or `/IMAGE`; with `/INCREMENTAL` the saveset must have one base.

Be careful with savesets you did not make yourself: `/ORIGINAL` writes wherever the saveset says.

Example

```
# vbackup /mnt/usb/home.bck /ORIGINAL /REPLACE
```

/OWNER

Output file qualifier.

Format

`/OWNER=ORIGINAL /OWNER=DEFAULT /OWNER=user`

Description

Specifies the owner of the restored or copied files.

Option	Meaning
`ORIGINAL`	The owner and group saved: by name when the name is known on this system, by number otherwise. The default for root.
`DEFAULT`	The files belong to the user who runs VBACKUP. The default for other users, who cannot give files away.
user	All files belong to that user and to his primary group.

`ORIGINAL` and `DEFAULT` may be abbreviated. An unknown user is reported with IVQUAL; an owner that cannot be set, with ATTRERR.

Example

```
# vbackup /backup/rrl.bck /home /OWNER=rrl
```

/PARITY

Output save-set qualifier.

Format

`/PARITY=m`

Description

Specifies the number of parity blocks that close every group, 1 to 8. With `/PARITY=1`, the default, a group ends with its XOR block and one bad block of a group is rebuilt. With *m* of 2 or more the XOR block is followed by *m* - 1 PARITY blocks (Reed-Solomon over GF(2^8), format.md 4.1): any *m* bad blocks of a group - adjacent ones too, DATA or parity - are rebuilt when the saveset is read (BLKFIXED). The saveset grows by *m*/*n* of its data, *n* being the group size. When more parity blocks of a group are good than are needed, the rows left over check the repair; a group whose parity does not agree is not rebuilt (PARITYERR).

A saveset with *m* of 2 or more has format version 2 in every block (3 when it may hold SOLID records, see `/SOLID`). VBACKUP and vbkx before X01-14 refuse it (NOTSAVESET);

vbkx-go, vbkx-rs, vbkx-pl and the WCX plugin of X01-14 read it. /PARITY needs groups: it is not valid with /GROUP_SIZE=0.

Example

```
$ vbackup /home /mnt/tape/home.bck /GROUP_SIZE=20 /PARITY=4
```

The saveset is 20% larger than its data and survives four bad blocks in a row in every group of twenty.

/PHYSICAL

Command qualifier.

Format

```
device|image-file saveset /PHYSICAL saveset device|image-file /PHYSICAL [/REPLACE]
```

Description

On a save, copies one block device or image file block by block (see Section 3.6). The input must be a block device or a regular file (PHYSNOTDEV). A device that is mounted read-write, or a partition of which is, is refused (PHYSMOUNTED); so is a device in use (PHYSHELD). If the device changes its size while it is read, PHYSSIZE is reported. The totals are reported by PHYSSUMM.

On a restore, the saveset must have been made with /PHYSICAL (PHYSNOTPHYS). Onto a block device, /REPLACE is required (PHYSREPLACE); a mounted device (PHYSMOUNTED), a busy device (PHYSHELD) and a device smaller than the one saved (PHYSSMALL) are refused; at a terminal YES is asked as for /IMAGE (PHYSABORT). Onto a larger device, PHYSLARGER is reported. A new image file is created; an existing one is overwritten only with /REPLACE. A lost block of the saveset is a lost part of the device (FILDAMAGED). After a restore onto a device, PHYSUUID reminds you that the copy carries the UUIDs of the original.

/PHYSICAL takes one input on a save and one saveset on a restore. It refuses /SELECT, /EXCLUDE, /SINCE, /BEFORE, /BY_OWNER, /RECORD, /INCREMENTAL, /IMAGE, /ORIGINAL and /DELETE. /DATA_FORMAT=COMPRESSED and /ENCRYPT work with it.

Example

```
# vbackup /dev/sdb1 /mnt/usb/sdb1.bck /PHYSICAL
# vbackup /mnt/usb/sdb1.bck /dev/sdc1 /PHYSICAL /REPLACE
```

/RECORD

Command qualifier.

Format

/RECORD /NORECORD (default)

Description

On a save, records the saveset in the journal after it has been written and, with /VERIFY, verified without a difference; and for every file saved with the status OK, its absolute name, inode, size, modification and change times (RECORDED). A file that changed while it was saved, or could not be read, is not recorded, so the next /SINCE=BACKUP saves it again. A saveset written to - is recorded as (standard output), one written to another node as *node*::*file*.

With savesets and no output specifier, rebuilds the journal from their catalogs: every entry with the status OK and inode data (X01-02 or later) gives a file state, the later saveset winning. A saveset without a TRAILER is skipped (NOTRAILER); entries without inode data are counted (NOINODE).

/RECORD cannot be combined with /PHYSICAL.

Example

```
$ vbackup /home full.bck /RECORD /JOURNAL=/backup/home.jnl
$ vbackup full.bck,mon.bck /RECORD /JOURNAL=/backup/home.jnl
%VBACKUP-I-STARTED, Operation: rebuild of the journal, Input: full.bck,mon.bck - started
%VBACKUP-I-RECORDED, Files: 6, Journal: /backup/home.jnl - recorded
%VBACKUP-I-COMPLETED, Operation: rebuild of the journal, Seconds: 0.00 - completed
```

/REPLACE

Output file qualifier; output save-set qualifier.

Format

/REPLACE /NOREPLACE (default)

Description

Operation	Effect
Save	An existing saveset is overwritten. Without /REPLACE the save stops with OPENOUT (errno 17, File exists). With a saveset on another node, /REPLACE is passed on to VBACKUP there.
Copy of a saveset	The existing volumes of the output are overwritten. Without /REPLACE the copy stops at the first volume that exists (OPENOUT, errno 17).
Restore, copy	An existing file is removed and restored again. Without /REPLACE it is kept, and FILEEXISTS is reported.
Extract	An existing output file is overwritten.
Restore /PHYSICAL, /IMAGE	The output device is overwritten; required.

/INCREMENTAL implies /REPLACE.

Example

```
$ vbackup /backup/rrl.bck /home '/SELECT=*.c' /REPLACE
```

/SAVE_SET

Output save-set qualifier.

Format

/SAVE_SET

Description

Specifies that the output specifier is a saveset, whatever its name. It is not needed when the name ends in .bck or .sav, or is -. Without it, an output with another name beside an input that is not a saveset means a copy. Beside an input that is a saveset, it makes the command a copy of the saveset (/TRANSFER). On the input side /SAVE_SET has no effect: a saveset is recognized by its contents.

Example

```
$ vbackup /etc /backup/etc.2026-10-05 /SAVE_SET
```

/SELECT

Input file-selection qualifier.

Format

`/SELECT=(pattern[,...])`

Description

Takes only the files whose stored name matches one of the patterns. In a pattern, `*` matches any characters including `/`, and `%` and `?` match one character. A stored name is relative to the base, for example `rrl/src/a.c`; `/LIST` shows the stored names. At most 64 patterns.

On a save and a copy, the patterns select files; directories are always taken. On a restore, a directory is restored only when it matches itself (the parents of a selected file are made anyway); on a comparison the files that do not match are skipped. With `/JOURNAL /LIST /FULL` the patterns are matched against the absolute names of the journal. On a saveset listing only the entries that match are listed. It cannot be combined with `/INCREMENTAL`, `/PHYSICAL` or `/IMAGE`.

Example

```
$ vbackup home.bck /tmp/r '/SELECT=(*.c,*.h)'
```

/SINCE

Input file-selection qualifier.

Format

`/SINCE=time /SINCE=BACKUP`

Description

Selects for saving only the files whose time is not earlier than the time given (see `/BEFORE` for the forms of a time). The time examined is the modification time, unless `/CREATED` or `/CHANGED` is given. Directories are always saved.

`/SINCE=BACKUP` selects the files that changed since they were last saved with `/RECORD`, as the journal knows (Section 3.5). A file the journal does not know is saved. Without a journal everything is saved, as the first time, and NOJOURNAL is reported.

Any `/SINCE` makes the saveset incremental; the summary keeps the filter as it was given (`Filter:` in the listing), and INCRSUMM counts the files listed as present. `/SINCE` cannot be combined with `/PHYSICAL` or `/DELETE`.

Example

```
$ vbackup /home /mnt/usb/home.bck /SINCE=TODAY /VOLUME_SIZE=4G
$ vbackup /home mon.bck /SINCE=BACKUP /RECORD
```

/SOLID

Output save-set qualifier.

Format

`/SOLID` (default when the save compresses) `/NOSOLID`

Description

With `/DATA_FORMAT=COMPRESSED` or `/LEVEL`, the small regular files – each of at most 256 KB, not a further name of a hard link - are gathered into groups of up to 256 KB (1 MB in X01-21), in the order of the walk, and each group is compressed as one SOLID record: the FILE, DATA and FEND records of its files together (format.md 6.12). The dictionary is shared and the metadata is compressed too, so a tree of small files gets much smaller: `/usr/include`

at /LEVEL=6 55.6 MB with /NOSOLID, 37 MB with /SOLID (Section 3.7). Big files, directories and links are written as before; each of them closes the open group first.

/SOLID is the default whenever the save compresses; /SOLID alone does not compress. /NOSOLID writes every file in records of its own, as before X01-21. It has no effect with /PHYSICAL; it works with /IMAGE, /ENCRYPT and /PARITY. Every SOLID is decompressed and compared before it is written; a group that does not come back the same (ZCHECK) or does not shrink is written as ordinary records.

A saveset that may hold SOLID records has format version 3 in every block. VBACKUP, vbkx and the plugins before X01-21 refuse it (NOTSAVESET); they never misread it. /TRANSFER copies it block for block, unchanged. A restore, /LIST, /COMPARE, /EXTRACT, /SELECT, /VERIFY, /DELETE, vbkx and the file manager plugins read SOLID records by themselves; a single file is reached through the catalog by its SOLID, which is decompressed, the files before it passed.

The cost: blocks lost beyond repair - more than *m* bad blocks in one group, *m* being the /PARITY - take the files of the SOLID they cut from the damage on, and every file of a SOLID that begins in them, not only the file they were in: a handful to a couple of dozen small files (Section 3.7). The files of the SOLID before the cut are restored, each checked by its FEND CRC (since X01-22, for savesets of X01-21 too); the file at the cut is named with FILDAMAGED, the files after it with FILLOST. With the default parity one bad block per group is still rebuilt, as always. For a saveset that must live through heavy damage, give /PARITY=2 or more, or /NOSOLID.

Examples

```
$ vbackup /home /mnt/usb/home.bck /LEVEL=6
$ vbackup /home /mnt/usb/home.bck /LEVEL=6 /PARITY=2
$ vbackup /home /mnt/usb/old.bck /LEVEL=6 /NOSOLID
```

The first saveset packs the small files of /home in SOLID records. The second does too, and survives two bad blocks in every group. The third is for a reader before X01-21: every file in records of its own, format version 1.

/TRANSFER

Command qualifier.

Format

```
input-saveset output-saveset /TRANSFER
```

Description

Copies a saveset to a saveset, block for block. The blocks are copied as they are - never the records: the copy is byte for byte the original, volume for volume, and an encrypted saveset is copied without its passphrase. The input is one saveset, given by its first volume, or -; the output is a saveset name, or -:

Command	Effect
vbackup x.bck y.bck	A copy, volume for volume: y.bck, y.bck.002, …
vbackup x.bck -	The volumes, back to back, to the standard output.
vbackup - y.bck	The stream of the standard input split into y.bck, y.bck.002, … at each VHDR.

Every block is checked on the way. A bad block is copied as it is and reported (BLKCOPIED): the copy keeps what it was given, and a restore repairs the block from its group. A missing volume of the input is reported (MISSVOL) and the others are still copied; an input that

ends before its TRAILER is reported (NOTRAILER). An input that does not begin with the VHDR of volume 1 is not taken (NOTSAVESET). The block size and the volumes of the input are kept. XFRSUMM gives the totals; `/LOG` reports volume 1 too (CREATED); `/REPLACE` overwrites volumes that exist.

A saveset input and a saveset output name mean this copy without the qualifier (Section 5.1). `/TRANSFER` asks for it whatever the names would mean: `vbackup x.bck dir /TRANSFER` writes the saveset `dir`, `dir.002`, ..., and restores nothing. With any other input or output it is refused (QUALUSE: `the input must be one saveset (or -), and there must be an output`). The commands VBACKUP gives to another node carry it, so that a VBACKUP there before X01-11 refuses them rather than restores (Section 1.10).

Example

```
$ vbackup /backup/home.bck /mnt/usb/home.bck /LOG
%VBACKUP-I-STARTED, Operation: copy of a saveset, Input: /backup/home.bck, Output: /mnt/usb/home.bck -
started
%VBACKUP-I-CREATED, Volume: /mnt/usb/home.bck - created
%VBACKUP-I-CREATED, Volume: /mnt/usb/home.bck.002 - created
...
%VBACKUP-I-CREATED, Volume: /mnt/usb/home.bck.005 - created
%VBACKUP-I-XFRSUMM, Blocks: 72, Volumes: 5, Bad: 0 - copied
%VBACKUP-I-COMPLETED, Operation: copy of a saveset, Seconds: 0.02 - completed
$ vbackup /backup/home.bck /TRANSFER
%VBACKUP-E-QUALUSE, Qualifier: /TRANSFER - the input must be one saveset (or -
), and there must be an output
```

/VERIFY

Command qualifier.

Format

`/VERIFY` `/NOVERIFY` (default)

Description

On a save, after the saveset has been written, reads it back and compares every file with the disk (VERIFYING, then COMPARERR for a difference and CMPSUMM for the totals). A file of the catalog that the stream did not give - lost in bad blocks, or never there - counts as a difference (since X01-22), so `/VERIFY` never says `Differences: 0` over a missing file. With a saveset written to the standard output it is refused (QUALUSE). A saveset on another node is read back from there once the other side has completed, and compared with the disk here (Section 1.10). `/DELETE` and, under `/VERIFY`, `/RECORD` act only when the verification found no difference.

On a copy, reads every regular file back from both sides as soon as it has been copied and compares them; a difference is reported with COMPARERR, and CMPSUMM gives the totals when differences were found.

Example

```
$ vbackup /home/rrl /backup/rrl.bck /VERIFY
%VBACKUP-I-STARTED, Operation: save, Input: /home/rrl, Output: /backup/rrl.bck - started
%VBACKUP-I-SAVESUMM, Files: 6, Bytes: 108902, Blocks: 5, Volumes: 1 - saved
%VBACKUP-I-VERIFYING, Saveset: /backup/rrl.bck - verifying
%VBACKUP-I-CMPSUMM, Files: 6, Differences: 0 - compared
%VBACKUP-I-COMPLETED, Operation: save, Seconds: 0.00 - completed
```

/VOLUME_SIZE

Output save-set qualifier.

Format

`/VOLUME_SIZE=size`

Description

Cuts the saveset into volumes of the size given, in bytes, optionally followed by K, M, G or T (binary multiples) and an optional B. The size is rounded down to a multiple of the block size and must be at least (group size + 3) x block size: 851968 bytes with the defaults. The volumes are named *name*, *name*`.002`, *name*`.003` and so on (Section 3.3); each volume after the first is reported (CREATED). Keep them together in one directory.

With the standard output as the saveset the volumes go into it back to back, each beginning with its VHDR (Section 3.9); a receiver (`vbackup -`*saveset*) or VBACKUP on another node (Section 3.10) splits them into volume files again. A copy of a saveset keeps the volumes of its input, whatever `/VOLUME_SIZE` says.

Example

```
$ vbackup /home home.bck /VOLUME_SIZE=4G
$ vbackup /home home.bck /VOLUME_SIZE=100K
%VBACKUP-E-IVQUAL, Value: 100K, Qualifier: /VOLUME_SIZE - is not valid
```

/XATTRS

Input file qualifier; output file qualifier.

Format

`/XATTRS` (default) `/NOXATTRS`

Description

Saves, restores and copies the extended attributes of the files: user attributes, POSIX ACLs (`system.posix_acl_access`, `system.posix_acl_default`), file capabilities and the SELinux label. Some of them can be restored only by root; those that cannot be set are reported with ATTRERR. `/NOXATTRS` neither saves nor restores them.

Example

```
$ vbackup /home/rrl rrl.bck /NOXATTRS
```

Environment Variables

The following environment variables affect VBACKUP. Variables marked *vbkx* are also honoured by vbkx; those marked *extractors* by vbkx-go, vbkx-rs and vbkx-pl.

Variable	Meaning
`VBACKUP_KEY_FILE`	The key file of an encrypted saveset when `/KEY_FILE` is not given (*vbkx*, *extractors*, plugins).
`VBACKUP_NOPROMPT`	`1` - never ask for a passphrase at the terminal; without a key file the result is NOKEY. Set by the file manager plugins (*vbkx*).
`VBACKUP_KDFITER`	The PBKDF2 iteration count of a save `/ENCRYPT`, at least 1000; values below are ignored. For tests only - do not lower it for real savesets. The count is stored in the saveset.

Variable	Meaning
VBACKUP_NOSIMD	1 - compute the parity of /PARITY by the portable code even where the processor has vector instructions for it (AVX2, SSSE3 on x86; NEON on aarch64); for trouble-shooting.
VBACKUP_NOHWCRYPTO	1 - compute SHA-256 by the portable code even where the processor has instructions for it (aarch64); for trouble-shooting.
VBACKUP_CTHREADS	The threads of the encryption and of the decompression of a restore, /COMPARE, /VERIFY (X01-23), the caller included: the number of processors by default, at most 8; 1 - none.
VBACKUP_ZTHREADS	The threads that compress a save /DATA_FORMAT=COMPRESSED: the number of processors by default, at most 8; 1 or less - none.
VBACKUP_PIPELINE	0 - the saveset is written without the writer thread, for trouble-shooting. The hints to the page cache stay; the saveset is the same.
VBACKUP_PREFETCH	The threads that read the next files ahead in a save or a copy: 8 by default, at most 64; 0 - none. More may help on NFS or a slow network disk, fewer on a single slow hard disk.
VBACKUP_RSH	The command that starts VBACKUP on another node for *node*::*file* (Section 3.10), in place of ssh. It is run as *command node remote-command*, the way ssh is.
VBACKUP_HELPLIB	The help library used by /HELP.
HOME	The directory of the default journal of a user other than root.

Chapter 6. Messages

6.1 Message Format

VBACKUP writes its messages to the standard error, one line each:

```
dd-mm-yyyy hh:mm:ss.cc pid %VBACKUP-s-IDENT, text
```

Field	Meaning
dd-mm-yyyy hh:mm:ss.cc	The date and time, to hundredths of a second
pid	The process number
`VBACKUP`	The facility name
s	The severity: S, I, W, E or F
`IDENT`	The message identifier, by which the message is listed below
text	The message text

The text has one form throughout: what the message is about first, as `Label: value` pairs - File, Saveset, Device, Volume, Block, Files, Bytes, errno - then a hyphen and the words, so that names and numbers are found at a glance:

```
%VBACKUP-I-BLKFIXED, Block: 3, Volume: 1 - was bad, rebuilt from its group
```

An *errno* value is followed by its text from the C library in parentheses.

6.2 Severities and the Exit Status

Severity	Meaning	Effect on the exit status
S - Success	The operation succeeded.	None (0).
I - Informational	A report; nothing is wrong.	None.
W - Warning	Something was not as it should be, but the operation was done; the output may need attention.	At least 1.
E - Error	Something was not done: a file, a block, a comparison, or the command itself.	2.
F - Fatal	The operation could not continue.	2.

The exit status of VBACKUP is 2 if any E or F message was signalled, else 1 if any W message was signalled, else 0.

Errors detected by the command language routines of StarLet, before VBACKUP looks at the command, are displayed with the prefix `%CLI_RTNS` rather than `%VBACKUP`, for example an ambiguous abbreviation or an illegal keyword value; the exit status is then 2:

```
%CLI_RTNS-F: Ambiguous qualifier 'RE'
%CLI_RTNS-E: Illegal or unrecognized keyword 'ZIP'
```

A word that begins with a slash but names no qualifier is a file name, not an error by itself; it usually leads to MAXPARM or IVOP.

vbkx and the extractors of last resort do not use this message facility; their diagnostics are described in Appendix C.

6.3 Messages

ATTRERR, File: *file* - *what* not restored, errno: *n* (*reason*)

Facility: VBACKUP. **Severity:** Warning.

Explanation: An attribute of a restored or copied file could not be set. *what* is `owner`, `protection`, `times`, `flags` (chattr flags), `attributes` (of a directory, set at the end of the restore), the name of an extended attribute, or `the removal` - a file that `/INCREMENTAL` was to delete could not be deleted. The data of the file is restored.

User Action: Most often the restore is not run by root. Restore as root, or give `/OWNER=DEFAULT`; give `/NOXATTRS` if the extended attributes are not wanted. Some attributes cannot be held by the target file system (for example, ACLs on vfat).

BADREC, Block: *block*, Volume: *volume* - an invalid record, skipped

Facility: VBACKUP. **Severity:** Warning.

Explanation: A record whose header or body makes no sense was found in the record stream, in or after the block given, and was skipped. The blocks around it had correct checksums; the record was written wrong or the saveset was not made by VBACKUP. The file the record belongs to is reported as damaged if data was lost.

A SOLID record (Section 3.7) that does not decompress, or holds a record that makes no sense, is reported with the block where the SOLID begins. All of it is skipped, like lost blocks: none of its files is restored from it, and each of them is named with FILLOST - never restored with wrong data. A SOLID only cut short by lost blocks, or by the end of a saveset whose save was interrupted, is not reported here: its part before the cut is restored (see FILLOST).

User Action: Check the files reported with FILDAMAGED and FILLOST. If the saveset was made by VBACKUP, report the problem with a copy of the saveset.

BLKCOPIED, Block: *block*, Volume: *volume* - is bad, copied as it is: a restore repairs it from its group

Facility: VBACKUP. **Severity:** Warning.

Explanation: A copy of a saveset (`/TRANSFER`, a receiver of a stream, VBACKUP on another node receiving a save) met a block with a wrong checksum. It is copied as it is: the copy keeps what it was given, and a restore of the copy repairs the block from its group, as it would from the original. *block* is the position of the block in the copy, counted from 0, and *volume* the volume of the output being written; when no volume is missing before the bad block, they are its block and volume numbers in the saveset.

User Action: None for the copy, while no other block of the same group is bad. If the block went bad on the way (a network, a pipe), copy the saveset again; if it is bad in the original, the medium may be failing: copy the saveset to another medium and test it with a restore or `/COMPARE`.

BLKFIXED, Block: *block*, Volume: *volume* - was bad, rebuilt from its group

Facility: VBACKUP. **Severity:** Informational.

Explanation: A block of the saveset had a wrong checksum (or, in an encrypted saveset, a wrong tag) and was the only bad block of its group; it was rebuilt from the other blocks of the group and its XOR block. Nothing is lost.

User Action: None for the files. The medium may be failing: copy the saveset to another medium.

BLKFORGED, Block: *block*, Volume: *volume* - is not what was written: its CRC is right, its authentication fails

Facility: VBACKUP. **Severity:** Warning.

Explanation: A block of an encrypted saveset has a correct checksum but its authentication tag does not match: it was changed on purpose (and its CRC made right again), or the medium does very odd things. The block is treated as a bad block: BLKFIXED follows if it could be rebuilt from its group, BLKLOST otherwise.

User Action: When BLKFIXED follows, the files are correct. Find out who could write to the saveset.

BLKLOST, Block: *block*, Volume: *volume* - is bad and cannot be rebuilt

Facility: VBACKUP. **Severity:** Error.

Explanation: A block of the saveset is bad, and its group holds another bad block (or the saveset has no XOR blocks), so it cannot be rebuilt. The reader resumes at the next good block. The files whose data lay in the lost blocks are reported with FILDAMAGED or FILLOST; all other files are correct.

User Action: Keep the files restored; restore the damaged ones from another saveset. Next time give a smaller `/GROUP_SIZE`, and keep two copies of important savesets.

CMPSUMM, Files: *n*, Differences: *n* - compared

Facility: VBACKUP. **Severity:** Informational.

Explanation: The totals of a comparison (`/COMPARE`, `/VERIFY`): the files compared and the files found different - among them the files of the catalog that the record stream did not give (COMPARERR `not compared`). Displayed without `/LOG`. After a copy `/VERIFY` it is displayed only when differences were found.

User Action: None. If differences are reported, see COMPARERR.

COMPARED, File: *file* - compared

Facility: VBACKUP. **Severity:** Informational.

Explanation: With `/LOG`, a file was compared with the disk and no difference was found.

User Action: None.

COMPARERR, File: *file - difference*

Facility: VBACKUP. **Severity:** Error.

Explanation: `/COMPARE`, or `/VERIFY` of a save or a copy, found a difference between the saveset (or the copy) and the disk. *difference* is one of: `not found on the disk`, `not a regular file on the disk`, `not a directory on the disk`, `not a symbolic link on the disk`, `the link points elsewhere`, `not a FIFO on the disk`, `not a device on the disk`, `the size differs`, `the file on the disk is shorter`, `the contents differ at octet n`, and for a copy `cannot be read back` or `the copy differs from the original`. Only the first difference of a file is reported.

Since X01-22 the catalog is set against the records read, and a file of the catalog that the record stream did not give is a difference too: `not compared: its records were lost in bad blocks` after lost blocks (see FILLOST), `not compared: the catalog has it, the record stream does not` when no block was lost (see NOTINSTREAM). So `/VERIFY` cannot say `Differences: 0` over files lost in bad blocks.

User Action: If the file was changed on purpose after the save, none. Otherwise save it again, or find out what changed it. For `not compared`, see FILLOST or NOTINSTREAM. Under `/VERIFY`, a difference keeps `/DELETE` from deleting anything and `/RECORD` from recording the save.

COMPLETED, Operation: *operation*, Seconds: *s.cc - outcome*

Facility: VBACKUP. **Severity:** Informational.

Explanation: The operation (save, restore, compare, extract, copy, rebuild of the journal, copy of a saveset) has ended, after the elapsed time given. *outcome* is `completed`, `completed with warnings` or `completed with errors`, in agreement with the exit status. Displayed without `/LOG`; a listing does not display it.

User Action: None; if the outcome is not `completed`, look at the warnings and errors above it.

CONFQUAL, Qualifiers: */qualifier*, */qualifier* - cannot be given together

Facility: VBACKUP. **Severity:** Error.

Explanation: Two qualifiers that exclude each other were given. The pairs are: `/INCREMENTAL` with `/SELECT` or `/EXCLUDE`; more than one of `/MODIFIED`, `/CREATED`, `/CHANGED`; `/FULL` with `/BRIEF`; `/PHYSICAL` with `/SINCE`, `/BEFORE`, `/RECORD`, `/INCREMENTAL`, `/BY_OWNER`, `/SELECT` or `/EXCLUDE`; `/IMAGE` with `/PHYSICAL`, `/INCREMENTAL`, `/BY_OWNER`, `/SELECT` or `/EXCLUDE`; `/ORIGINAL` with `/PHYSICAL` or `/IMAGE`; `/DELETE` with `/PHYSICAL`, `/IMAGE`, `/SINCE` or `/BEFORE`; `/IMAGE` with `/CONFIRM` on a restore. Nothing was done.

User Action: Remove one of the qualifiers.

COPIED, File: *file* - copied

Facility: VBACKUP. **Severity:** Informational.

Explanation: With `/LOG`, a file was copied by a copy operation.

User Action: None.

CPYSUMM, Files: *n*, Bytes: *n* - copied

Facility: VBACKUP. **Severity:** Informational.

Explanation: The totals of a copy: the files made and the data bytes written. Displayed without `/LOG`.

User Action: None.

CRCERR, File: *file* - checksum mismatch: the data differ from what was saved

Facility: VBACKUP. **Severity:** Error.

Explanation: The data of a restored or extracted file does not match the CRC recorded when it was saved, although every block was read correctly. The file was written, but its contents are not what was saved. A saveset with compressed data read by VBACKUP before X01-04 gives this message as well.

User Action: Do not trust the file; restore it from another saveset. Use a current version of VBACKUP. If the saveset was made by VBACKUP and read by the same version, report the problem.

CREATED, Volume: *volume* - created

Facility: VBACKUP. **Severity:** Informational.

Explanation: A volume of the saveset was created. Volume 1 is reported under `/LOG` only; every further volume always. The further volumes of a saveset written to the standard output are reported as `Volume: (standard output)`: they follow one another in the stream.

User Action: None. When the volumes go to removable media, keep them together.

DELETED, File: *file* - deleted: it is not in the incremental saveset

Facility: VBACKUP. **Severity:** Informational.

Explanation: A restore `/INCREMENTAL` deleted a file from a directory of the saveset because the catalog of the saveset does not list it: the file had been deleted before that save. Displayed under `/LOG`.

User Action: None.

DELSUMM, Deleted: *n*, Kept: *n*

Facility: VBACKUP. **Severity:** Informational.

Explanation: The totals of `/DELETE`: the files deleted after the save and the verification, and the files kept.

User Action: If files were kept, see SRCKEPT.

ENCRYPTED, Saveset: *saveset*, Iterations: *n* - encrypted: ChaCha20, HMAC-SHA256, PBKDF2

Facility: VBACKUP. **Severity:** Informational.

Explanation: With `/LOG`, the saveset is being written encrypted; *n* is the PBKDF2 iteration count of its keys.

User Action: None. If *n* is below 600000, `VBACKUP_KDFITER` is set: remove it for real savesets.

FATALSAVE, Saveset: *saveset* - could not be completed

Facility: VBACKUP. **Severity:** Fatal.

Explanation: The saveset could not be written to its end: a volume could not be written (WRITERR precedes), the catalog could not be written, or memory ran out. The saveset is left as it is, without a TRAILER; it is not verified, and nothing is recorded or deleted.

User Action: Correct the cause reported before (most often a full disk) and save again. The files written up to the break can be restored from the incomplete saveset.

FILCHANGED, File: *file* - changed while it was being saved

Facility: VBACKUP. **Severity:** Warning.

Explanation: The size, modification time or change time of a file differed after it had been read from what they were before: the file was written to while it was saved, and the copy may be a mix of old and new. The file is saved with the status CHANGED and is not recorded in the journal, so the next `/SINCE=BACKUP` saves it again. A copy reports it the same way.

User Action: Save the file again when nobody writes to it, or save from a snapshot (LVM, btrfs).

FILDAMAGED, File: *file* - is incomplete: its data was lost in bad blocks

Facility: VBACKUP. **Severity:** Error.

Explanation: Part of the data of a file lay in blocks that were lost (BLKLOST, MISSVOL); the file was restored, compared or extracted as far as its data could be read. In a `/PHYSICAL` restore, the device has lost parts. Since X01-22 it also names the file at which lost blocks, or the end of an interrupted saveset, cut a SOLID record (Section 3.7): the files before it in the SOLID are restored whole, it is restored as far as it came, the files after it are FILLOST. A saveset with compressed data read by VBACKUP before X01-04 gives this message as well.

User Action: Restore the file from another saveset. All files not named in a message are correct.

FILEEXISTS, File: *file* - already exists, not restored

Facility: VBACKUP. **Severity:** Warning.

Explanation: A file to be restored, copied or extracted exists, and `/REPLACE` was not given; the existing file was kept.

User Action: Give `/REPLACE` to overwrite it, or restore into another directory.

FILLOST, File: *file* - not restored: its records were lost in bad blocks

Facility: VBACKUP. **Severity:** Error.

Explanation: The catalog lists a file that never came out of the stream: its FILE record was in lost blocks. The file is missing from the output.

Many FILLOST at once after one bad area (BLKLOST, or BADREC) mean that a SOLID record was hit (Section 3.7): the small files of a compressed save are kept in groups of up

to 256 KB (1 MB in X01-21), and lost blocks take the files of the group from the damage on, and all the files of a group that begins in them - a handful to a couple of dozen, not only one. Since X01-22 the files of a group before the damage are restored, each checked by its FEND CRC, for savesets of X01-21 too; the file at the cut is named with FILDAMAGED. Every lost one is named here; the other files are restored.

`/COMPARE` and `/VERIFY` report such a file with COMPARERR `not compared: its records were lost in bad blocks`, and count it as a difference.

User Action: Restore the files from another saveset. For savesets that must survive heavy damage, save next time with `/PARITY=2` or more, or with `/NOSOLID`.

GLUED, Parameter: *word* - the qualifiers glued to it are taken as qualifiers

Facility: VBACKUP. **Severity:** Informational.

Explanation: A parameter had qualifiers attached to it, as in `x.sav/sav/log`; VBACKUP took them apart as DCL does. The message is displayed so that a mistyped file name is not taken for a qualifier unnoticed.

User Action: None, if that was meant. If the word was a file name, give it as it is (a name of an existing file is never cut) or separate the qualifiers with spaces.

IMGCMD, Command: *command*

Facility: VBACKUP. **Severity:** Informational.

Explanation: With `/LOG`, a restore `/IMAGE` is about to run the command that makes the new file system.

User Action: None.

IMGMKFS, Command: *command* - failed: *reason*

Facility: VBACKUP. **Severity:** Error.

Explanation: The command that makes the new file system of a restore `/IMAGE` failed. *reason* is `the program is not installed`, the last line the program wrote, `it failed`, or the reason the program could not be started. Nothing was restored.

User Action: Install the program (e2fsprogs, xfsprogs, btrfs-progs, dosfstools), or correct the cause it reports, and restore again.

IMGMOUNT, Type: *type*, Device: *device*, errno: *n* - the new file system cannot be mounted (*reason*)

Facility: VBACKUP. **Severity:** Error.

Explanation: The file system just made on the device by a restore `/IMAGE` could not be mounted on its temporary directory. Nothing was restored.

User Action: Check that the kernel supports the file system type and that VBACKUP runs as root.

IMGNOID, Volume: *mount-point* - the *UUID* of its file system is not known: the new one gets a new one

Facility: VBACKUP. **Severity:** Warning.

Explanation: A save `/IMAGE` could not find the UUID of the file system (neither in `/dev/disk/by-uuid` nor, for ext file systems, in the superblock). The saveset is made; a restore `/IMAGE` will make a file system with a new UUID.

User Action: If the UUID matters (for example, `/etc/fstab` refers to it), set it on the new file system after the restore.

IMGNOTIMG, Saveset: *saveset* - was not made with /IMAGE: restore it without /IMAGE

Facility: VBACKUP. **Severity:** Error.

Explanation: A restore `/IMAGE` was given a saveset that holds no file system identity.

User Action: Restore the saveset without `/IMAGE`, into a directory.

IMGNOTMNT, Device: *device* - is not mounted: mount it (read-only is enough) and give the mount point or the device

Facility: VBACKUP. **Severity:** Error.

Explanation: The input of a save `/IMAGE` is a block device that is not mounted; VBACKUP reads the files through the mounted file system.

User Action: Mount the device, read-only if you like, and save again.

IMGNOTVOL, File: *file* - is neither the mount point of a file system nor a device: /IMAGE saves a whole volume

Facility: VBACKUP. **Severity:** Error.

Explanation: The input of a save `/IMAGE` is not the mount point of a file system (for example, a subdirectory) nor a mounted block device.

User Action: Give the mount point, or save the directory without `/IMAGE`.

IMGSMALL, Device: *device*, Bytes: *n*, Needed: *n* - too small, nothing written

Facility: VBACKUP. **Severity:** Error.

Explanation: The output device of a restore `/IMAGE` is smaller than the space the saved file system used, plus 5%, plus 16 MB for the metadata of the new file system.

User Action: Use a larger device.

IMGSUMM, Device: *device*, Type: *type*, Files: *n*, Bytes: *n* - file system made, files restored

Facility: VBACKUP. **Severity:** Informational.

Explanation: The totals of a restore `/IMAGE`: the file system was made on the device and the files were restored into it.

User Action: None.

IMGUNSUPP, Saveset: *saveset*, Type: *type* - VBACKUP does not make such a file system: use /PHYSICAL for it

Facility: VBACKUP. **Severity:** Error.

Explanation: The saveset holds a file system of a type for which VBACKUP cannot make a new file system (types made: ext2, ext3, ext4, xfs, btrfs, vfat, msdos).

User Action: Restore the files into a directory without `/IMAGE`, or save such volumes with `/PHYSICAL`.

INCRSUMM, Files: *n* - unchanged, listed as present, not saved

Facility: VBACKUP. **Severity:** Informational.

Explanation: An incremental save (a time filter was given) listed *n* covered files in its catalog as present without saving them.

User Action: None.

IVOP, cannot tell what to do: *reason*

Facility: VBACKUP. **Severity:** Error.

Explanation: The operation cannot be determined from the parameters: the input does not exist (and so is not a saveset), and the output is not the name of a saveset.

User Action: Check the input specification. To save, end the output name with `.bck` or `.sav`, or give `/SAVE_SET`.

IVQUAL, Value: *value*, Qualifier: /*qualifier* - is not valid

Facility: VBACKUP. **Severity:** Error.

Explanation: The value of a qualifier is not legal: `/BLOCK_SIZE` outside 8192 to 1048576 or not a multiple of 512; `/GROUP_SIZE` above 100; `/VOLUME_SIZE` not a size or less than (group size + 3) x block size; `/BY_OWNER` or `/OWNER` naming an unknown user; `/DATA_FORMAT` with an unknown keyword.

User Action: Correct the value.

IVTIME, Time: *value*, Qualifier: /*qualifier* - is not a valid time

Facility: VBACKUP. **Severity:** Error.

Explanation: The value of `/SINCE` or `/BEFORE` is not a time VBACKUP understands.

User Action: Write the time as described under `/BEFORE` in Section 5.2.

JNLERR, Journal: *journal* - *what*, errno: *n* (*reason*)

Facility: VBACKUP. **Severity:** Error.

Explanation: The journal cannot be used. *what* is one of: `no journal given, and no home directory`; `cannot make its directory`; `cannot be opened` (the journal or its lock file); `cannot be locked`; `cannot be read, or is too short`; `is no journal, or is damaged`; `cannot be created, cannot be written` (the temporary file); `cannot be replaced`. A save with `/SINCE=BACKUP` or `/RECORD` that cannot open the journal is not made.

User Action: Correct the cause given by *reason*. A damaged journal can be rebuilt: delete it and give the savesets with `/RECORD`.

KEYFILE, Key file: *file* - *reason*

Facility: VBACKUP. **Severity:** Error.

Explanation: The passphrase cannot be taken from the key file, or from the terminal (*file* is then `/dev/tty`). *reason* is the reason the file could not be opened or read; `not a regular file`; `others may read or change it - chmod 600 it`; `its first line is longer than 1024 bytes`; `its first line is empty`; or, for the terminal, `an empty passphrase` or `longer than 1024 bytes, or no line`. Nothing was read or written.

On Windows the mode of the key file is not looked at: its ACL decides who may read it.

User Action: Make the key file a regular file of its owner only (`chmod 600 file`) with the passphrase in its first line.

KEYMATCH, the two passphrases differ: nothing saved

Facility: VBACKUP. **Severity:** Error.

Explanation: A save `/ENCRYPT` asked for the passphrase twice at the terminal, and the two answers differ. No saveset was made.

User Action: Save again and type the same passphrase twice.

MAXPARM, Parameter: *word* - one too many: only an input and an output are taken; a qualifier begins with /

Facility: VBACKUP. **Severity:** Error.

Explanation: The command has more than two parameters. Most often a qualifier was typed without its slash (`.log` for `/LOG`), or a list of inputs contains spaces. Nothing was done.

User Action: Correct the command; separate several inputs by commas without spaces.

MISSING, File: *file* - should be there from an earlier saveset, and is not

Facility: VBACKUP. **Severity:** Warning.

Explanation: During a restore `/INCREMENTAL`, the catalog lists a file as present (unchanged since an earlier save), but no earlier saveset of the chain put it into the output.

User Action: Most often a saveset of the chain was left out. Give them all, the full one first, then the incremental ones oldest first.

MISSVOL, Volume: *n*, Saveset: *saveset* - is missing

Facility: VBACKUP. **Severity:** Error.

Explanation: A volume of the saveset was not found beside the first one, or a stream of several volumes passed from one volume to a later one (the saveset is then named -). The files that lie wholly in the other volumes are still restored; the others are reported. A copy of a saveset reports the volume it cannot find and copies the others.

User Action: Put all volumes into one directory, with their names unchanged, and repeat the operation. For a stream, find why the sender left the volume out (its own MISSVOL).

NOCATALOG, Saveset: *saveset* - has no catalog: it is listed by reading it whole

Facility: VBACKUP. **Severity:** Warning.

Explanation: `/LIST` found no usable catalog (the saveset has no TRAILER, or its catalog cannot be read) and lists the FILE records of the stream instead, which takes longer. Not displayed for a saveset read from a pipe.

User Action: None for the listing. See NOTRAILER.

NOFILES, Pattern: *specification* - no file matches it

Facility: VBACKUP. **Severity:** Warning.

Explanation: An input specification of a save or a copy selected no file, or its base could not be opened (OPENIN precedes).

User Action: Check the specification and its wildcards; quote the wildcards so that the shell does not expand them.

NOINODE, Saveset: *saveset*, Files: *n* - not recorded: the catalog has no inode data (written before X01-02)

Facility: VBACKUP. **Severity:** Warning.

Explanation: A journal rebuild found *n* catalog entries without the inode and change time needed for the journal; the saveset was written before X01-02. These files are not recorded.

User Action: None; the next `/SINCE=BACKUP` saves these files again.

NOJOURNAL, Journal: *journal* - is not there yet: /SINCE=BACKUP saves everything

Facility: VBACKUP. **Severity:** Informational.

Explanation: A save `/SINCE=BACKUP` found no journal; every covered file is saved, as the first time.

User Action: None if this is the first save of the plan. Otherwise check `/JOURNAL` and the user: every user has a journal of his own.

NOKEY, Saveset: *saveset* - needs a passphrase, and there is no terminal to ask it on: give /KEY_FILE=file or VBACKUP_KEY_FILE

Facility: VBACKUP. **Severity:** Error.

Explanation: The saveset is encrypted (or is to be made so), no key file was given, and there is no terminal to ask on (cron, a pipe, a file manager), or `VBACKUP_NOPROMPT=1` forbids asking.

User Action: Give `/KEY_FILE=file` or set `VBACKUP_KEY_FILE`.

NOMEM, errno: *n* - cannot allocate memory (*reason*)

Facility: VBACKUP. **Severity:** Fatal.

Explanation: Memory could not be allocated. The operation stops; a save in progress ends with FATALSAVE.

User Action: Free memory, or raise the limits of the process, and repeat the operation.

NOPARAM, Parameter: *parameter* - is missing

Facility: VBACKUP. **Severity:** Error.

Explanation: A required parameter was not given: the input specification, the output specification, the output directory of a restore, or the output device of a restore `/PHYSICAL`.

User Action: Give the parameter. To restore a saveset where its files came from, give `/ORIGINAL`.

NORMAL, normal successful completion

Facility: VBACKUP. **Severity:** Success.

Explanation: The success condition of the facility. VBACKUP does not display it; success is shown by COMPLETED and the exit status 0.

User Action: None.

NOTFOUND, File: *name* - is not in the saveset

Facility: VBACKUP. **Severity:** Error.

Explanation: `/EXTRACT` names a file that the saveset does not hold. The name must be exact, as `/LIST` shows it, without wildcards.

User Action: List the saveset and give the stored name.

NOTINCR, Saveset: *saveset* - *reason*: nothing restored with /INCREMENTAL

Facility: VBACKUP. **Severity:** Error.

Explanation: A restore `/INCREMENTAL` refused a saveset because it cannot trust its list of files: `it does not say whether it is full or incremental (written before X01-02)`, `it has no catalog`, `its catalog cannot be read`, or `its catalog is damaged`. Nothing is restored from it, and the chain ends there.

User Action: Restore that saveset without `/INCREMENTAL`. Savesets made by X01-02 and later carry what `/INCREMENTAL` needs.

NOTINSTREAM, File: *file* - not restored: the catalog has it, the record stream does not, though no block was lost

Facility: VBACKUP. **Severity:** Error.

Explanation: After the record stream was read, the catalog was set against the FILE records met (since X01-22, always, not only after lost blocks), and it names a file whose records never came - while no block was lost. The file is missing from the output. It does not happen with a saveset VBACKUP wrote right and nobody changed: the stream holds a record this reader does not know (for example a record of a newer VBACKUP in a saveset of an older format version), the saveset was edited or forged, or it was made wrong. From a pipe the catalog records are taken as they come. The completion is that of an error, as with FILLOST. `/COMPARE` and `/VERIFY` report the same file with COMPARERR `not compared: the catalog has it, the record stream does not`; vbkx says `File: `*file*` - not extracted: the catalog has it, the record stream does not`.

User Action: Try a newer VBACKUP. Otherwise restore the file from another saveset, and report it, with the saveset.

NOTOPIC, Topic: *topic* - sorry, no documentation on it

Facility: VBACKUP. **Severity:** Error.

Explanation: `/HELP` names a topic that the help library does not have.

User Action: Give `vbackup /HELP` to see the topics.

NOTRAILER, Saveset: *saveset* - has no trailer: the save did not complete, or its last volume is missing

Facility: VBACKUP. **Severity:** Warning.

Explanation: The last block of the saveset is not its TRAILER: the save was interrupted (the disk became full, the program was stopped), the last volume is missing, or a pipe ended before the saveset did - with a stream of several volumes, before its last volume came. The files up to the break can be restored; the saveset is read sequentially. A journal rebuild skips such a saveset. A copy of a saveset reports it after it has copied what came, and its copy has no TRAILER either.

User Action: Find the missing volume if there is one. Otherwise restore what can be restored and make a new saveset.

NOTSAVESET, File: *file* - is not a saveset

Facility: VBACKUP. **Severity:** Error.

Explanation: `/LIST`, `/COMPARE` or `/EXTRACT` was given an input that is not a saveset, or more than one input; or a file to be read as a saveset is not one; or the input of a copy of a saveset does not begin with the volume header of volume 1. A VBACKUP, vbkx or plugin before X01-21 says it of a saveset of format version 3, one written with compression by X01-21 or later without `/NOSOLID` (see `/SOLID`); before X01-14, of one of version 2 (`/PARITY=2` or more). With *node*`::`*file* the file is `-`: VBACKUP there sent nothing, and its messages above say why.

User Action: Give the first volume of one saveset. For a saveset of version 3 or 2, use VBACKUP X01-21 or later; for an older reader, save with `/NOSOLID` (and without `/PARITY` of 2 or more).

OPENDIR, Directory: *directory*, errno: *n* - cannot be read (*reason*)

Facility: VBACKUP. **Severity:** Error.

Explanation: A directory met by a save or a copy could not be opened or read; what it holds is not saved.

User Action: Run the save as a user who can read the directory (root), or exclude it.

OPENIN, File: *file*, errno: *n* - cannot be opened as input (*reason*)

Facility: VBACKUP. **Severity:** Error.

Explanation: A file could not be opened for reading: a file or the base of a specification during a save or a copy, a saveset, a device of a `/PHYSICAL` save, the input of a save `/IMAGE`, a file being compared; or the name is too long (errno 36). When the input of a command does not exist and no output is given, OPENIN reports it with errno 2.

User Action: Check the name and the permissions.

OPENOUT, File: *file*, errno: *n* - cannot be created as output (*reason*)

Facility: VBACKUP. **Severity:** Error.

Explanation: A file could not be created: a volume of the saveset (errno 17, File exists, when it exists and `/REPLACE` was not given), a restored, copied or extracted file, the listing file of `/LIST=file`, the temporary catalog spool, the output of a `/PHYSICAL` or `/IMAGE` restore. With errno 22 and the reason `a name that leads out of the output directory`, the saveset holds a name with `..` or a leading `/`; such a file is never written. A name whose way leads through a symbolic link is refused in the same way. On Windows, errno 22 with `not a valid name on Windows` is a name Windows cannot hold (section 1.17), and errno 17 with `a name that differs in case only is there` a second name of one file there.

User Action: Check the permissions and the free space; give `/REPLACE` to overwrite a saveset. A name that leads out of the output directory means that the saveset was not made by VBACKUP, or was made to do harm.

ORIGNOBASE, Saveset: *saveset* - does not say where its files came from (made before X01-02): give an output directory

Facility: VBACKUP. **Severity:** Error.

Explanation: A restore `/ORIGINAL` was given a saveset that does not record its bases as absolute names.

User Action: Restore the saveset into an output directory.

ORIGTARGET, Saveset: *saveset*, Target: *directory* - its files go back there

Facility: VBACKUP. **Severity:** Informational.

Explanation: A restore `/ORIGINAL` is about to put the files of the saveset back under the directory given; one message per base.

User Action: None; press Ctrl/C at once if that is not the place.

PARITYERR, Block: *n*, Volume: *n* - the group beginning here does not agree with its parity: a block of it with a right CRC holds other bytes, nothing of it is rebuilt

Facility: VBACKUP. **Severity:** Warning.

Explanation: A saveset made with `/PARITY`: blocks of the group that begins at the block named were bad, and every way of rebuilding them from its parity blocks left a parity row that disagrees - some block of the group whose CRC is right holds other bytes than were written. Nothing of the group is rebuilt; its bad blocks are lost (BLKLOST), and the files in them are reported (FILDAMAGED).

User Action: Keep the saveset; the other groups are not affected. Find out how the medium came to hold such a block.

PHYSABORT, Device: *device* - not overwritten: the answer was not YES

Facility: VBACKUP. **Severity:** Error.

Explanation: A restore `/PHYSICAL` or `/IMAGE` onto a device asked at the terminal for `YES`, and the answer was something else. Nothing was written.

User Action: None, if the device was wrong. Otherwise repeat and answer `YES`, in capitals.

PHYSHELD, Device: *device* - is in use (*holder*): free it first, or save what uses it

Facility: VBACKUP. **Severity:** Error.

Explanation: The device of a `/PHYSICAL` save, or the output device of a `/PHYSICAL` or `/IMAGE` restore, is in use: `held by` a device-mapper or RAID device (an LVM physical volume, a RAID member, a dm-crypt container), used as `swap`, or `the kernel says it is busy`.

User Action: Free the device (deactivate the volume group, stop the array, close the container, `swapoff`), or save the device that uses it.

PHYSLARGER, Device: *device*, Bytes: *n*, Saved: *n* - larger: the rest stays as it is, the file system keeps its old size

Facility: VBACKUP. **Severity:** Informational.

Explanation: The output device of a restore `/PHYSICAL` is larger than the device saved. The image is written to its first part; the rest is not touched.

User Action: Grow the file system if you want the space (`resize2fs`, `xfs_growfs`, ...).

PHYSMOUNTED, Device: *device* - is mounted[read-write] on *mount-point*: *advice*

Facility: VBACKUP. **Severity:** Error.

Explanation: A save `/PHYSICAL` was given a device that is mounted read-write (*advice*: `unmount it, mount it read-only, or save a snapshot`), or a restore `/PHYSICAL` or `/IMAGE` was given an output device that is mounted (*advice*: `nothing is written to a mounted device, unmount it`). *device* may say `(a partition of it)`. A device written to while it is copied gives a broken copy, which looks fine until it is needed.

User Action: Follow the advice.

PHYSNOTDEV, File: *file* - is neither a block device nor a file

Facility: VBACKUP. **Severity:** Error.

Explanation: The input of a `/PHYSICAL` save, or the output of a `/PHYSICAL` restore, is neither a block device nor a regular file; or the output of a restore `/IMAGE` is not a block device.

User Action: Give a block device or an image file.

PHYSNOTPHYS, Saveset: *saveset* - was not made with /PHYSICAL: restore it without /PHYSICAL

Facility: VBACKUP. **Severity:** Error.

Explanation: A restore `/PHYSICAL` was given a saveset that does not hold a device.

User Action: Restore it without `/PHYSICAL`.

PHYSREPLACE, Device: *device* - everything on it would be overwritten: give /REPLACE to do so

Facility: VBACKUP. **Severity:** Error.

Explanation: A restore `/PHYSICAL` or `/IMAGE` onto a block device was given without `/REPLACE`. Nothing was written.

User Action: Check that the device is the right one, then give `/REPLACE`.

PHYSSIZE, Device: *device*, Bytes: *n*, then: *n* - changed its size while it was read: the copy is not to be trusted

Facility: VBACKUP. **Severity:** Error.

Explanation: The size of the device of a `/PHYSICAL` save was different at the end of the copy from what it was at the beginning.

User Action: Find out what changed the device (a resize, a hot-plug), and save it again.

PHYSSMALL, Device: *device*, Bytes: *n*, Saved: *n* - too small, nothing written

Facility: VBACKUP. **Severity:** Error.

Explanation: The output device of a restore `/PHYSICAL` is smaller than the device saved.

User Action: Use a device at least as large; or restore into an image file, or with `/IMAGE` if the saveset is one.

PHYSSUMM, Device: *device*, Bytes: *n*, Data: *n* - the rest zeros

Facility: VBACKUP. **Severity:** Informational.

Explanation: The totals of a `/PHYSICAL` save or restore: the size of the device and the bytes of data that were not zeros.

User Action: None.

PHYSUUID, Device: *device* - now carries the labels and UUIDs of the device saved: never mount it beside the original

Facility: VBACKUP. **Severity:** Informational.

Explanation: A restore `/PHYSICAL` onto a device, or a restore `/IMAGE` that kept the UUID, has made a device that looks like the original to the system.

User Action: Do not mount the copy and the original at the same time: xfs refuses it, btrfs can damage both. Change the UUID of one of them if both must be used.

QUALUSE, Qualifier: */qualifier* - *reason*

Facility: VBACKUP. **Severity:** Error.

Explanation: A qualifier was given where it cannot be used; *reason* says why: `/DELETE` without `/VERIFY`; `/ENCRYPT` on an operation other than a save; `/VERIFY` or `/LIST` with the standard output as the saveset (`a saveset written to the standard output is gone once written: it cannot be read back here`); `/DELETE` or `/LIST` with a saveset on another node (`not with`

`a saveset made on another node`); `/TRANSFER` when the command is not a copy of a saveset (`the input must be one saveset (or -), and there must be an output`); `/PHYSICAL` or `/IMAGE` with more than one input (`one input - one device or volume - a saveset`) or more than one saveset (`one saveset at a time`); `/ORIGINAL` with an output specifier, or with `/INCREMENTAL` and a saveset of several bases.

On Windows, `not on Windows` is said of `/PHYSICAL` and `/IMAGE`: `vbackup.exe` has neither (section 1.17).

User Action: Correct the command as the reason says.

READERR, File: *file*, errno: *n* - cannot be read (*reason*)

Facility: VBACKUP. **Severity:** Error.

Explanation: A read failed: a file being saved (it is saved with the status READERR up to the failure, and is not recorded in the journal), a file being copied or compared, or the temporary catalog spool of a save.

User Action: Check the device and the file; save the file again.

RECORDED, Files: *n*, Journal: *journal* - recorded

Facility: VBACKUP. **Severity:** Informational.

Explanation: The journal has been updated by a save `/RECORD` (*n* files recorded), or rebuilt from catalogs.

User Action: None.

REMOTE, Node: *node*, errno: *n* - the pipe to VBACKUP there cannot be made (*reason*)

Facility: VBACKUP. **Severity:** Error.

Explanation: For a saveset on another node, *node*::*file*, VBACKUP could not make the pipe to VBACKUP there, nor start the process for ssh (or for the command of `VBACKUP_RSH`). Nothing was sent or read. A command that cannot be run, or a node that cannot be reached, is not reported here but by REMOTEERR.

On Windows, errno 40 with `not on Windows yet: vbackup.exe` does not reach savesets on other nodes in this version.

User Action: Most often the system is short of processes or file descriptors: look at *reason*, and repeat the operation.

REMOTEERR, Node: *node*, Exit: *n* - VBACKUP there did not complete: see its messages above

Facility: VBACKUP. **Severity:** Error.

Explanation: VBACKUP on the other node, or ssh that runs it, ended with the completion code *n* (2 - VBACKUP there signalled an error; 127 – the command of `VBACKUP_RSH` cannot be run here, or `vbackup` is not on the PATH there; 255 - ssh failed: the node cannot be reached, or does not let this one in), or was ended by a signal (*n* = 128 + the signal). A code of 1 (warnings) is not reported: it makes the exit status at least 1. When the other side stopped during a save, WRITERR (errno 32, Broken pipe) and FATALSAVE come before this message.

User Action: Read the messages of the other side above it. Check that `ssh`*node*`vbackup` displays the usage text without a question, that VBACKUP there is X01-11 or later (an earlier one refuses `/TRANSFER`), and that the saveset there can be created or read.

RESTORED, File: *file* - restored

Facility: VBACKUP. **Severity:** Informational.

Explanation: With `/LOG`, a file was restored.

User Action: None.

RESTSUMM, Files: *n*, Bytes: *n* - restored

Facility: VBACKUP. **Severity:** Informational.

Explanation: The totals of a restore: the files and directories made and the data bytes written, over all savesets of the command. Displayed without `/LOG`.

User Action: None.

SAVED, File: *stored-name* - saved

Facility: VBACKUP. **Severity:** Informational.

Explanation: With `/LOG`, a file was saved.

User Action: None.

SAVESUMM, Files: *n*, Bytes: *n*, Blocks: *n*, Volumes: *n* - saved

Facility: VBACKUP. **Severity:** Informational.

Explanation: The totals of a save: the files saved, the data bytes saved, the blocks of all types written, the volumes. Displayed without `/LOG`.

User Action: None.

SKIPPED, File: *stored-name* - skipped: *reason*

Facility: VBACKUP. **Severity:** Informational.

Explanation: With `/LOG`, a file or directory was left out of a save or a copy; *reason* is `nodump flag set`.

User Action: None. To save such files, give `/IGNORE=NOBACKUP`.

SRCDELETED, File: *file* - deleted: it is in the saveset and verified

Facility: VBACKUP. **Severity:** Informational.

Explanation: With `/DELETE /LOG`, a file was deleted after it had been saved and verified.

User Action: None.

SRCKEPT, File: *file* - not deleted: *reason*

Facility: VBACKUP. **Severity:** Warning.

Explanation: `/DELETE` kept a file. *reason* is `it changed after it was saved`, the reason the deletion failed, or - with *file* `every file – the saveset did not verify`.

User Action: Save the file again if it changed; delete it by hand if that is wanted.

STARTED, Operation: *operation*, Input: *input*[, Output: *output*] - started

Facility: VBACKUP. **Severity:** Informational.

Explanation: An operation begins. *operation* is `save`, `restore`, `compare`, `extract`, `copy`, `rebuild of the journal` or `copy of a saveset`. The standard input and output are shown as `(standard input)` and `(standard output)`; the output of a restore `/ORIGINAL` as `(where its files came from)`; a saveset on another node as *node*`::`*file*. When several input specifications are given, all are shown as they were given. Displayed without `/LOG`; a listing does not display it. VBACKUP on another node displays its own STARTED, with the operation `copy of a saveset`.

User Action: None.

TOODEEP, Directory: *directory*, Levels: *n* - nested deeper, not descended

Facility: VBACKUP. **Severity:** Warning.

Explanation: A directory met by a save or a copy is nested deeper than 256 levels below the base; what it holds is not saved.

User Action: Save that directory with a specification of its own.

TOOMANY, Too many: *what*, Taken: *n* - the rest is ignored

Facility: VBACKUP. **Severity:** Warning.

Explanation: More than 64 input specifications, or more than 64 patterns of `/SELECT` or `/EXCLUDE`, were given; the first 64 are taken.

User Action: Use fewer specifications or patterns: a wildcard, a common directory, a broader pattern.

UNNAMED, Saveset: *saveset* - blocks were lost and *reason*: files missing from the restore cannot all be named

Facility: VBACKUP. **Severity:** Warning.

Explanation: Blocks were lost during a restore, and the saveset has no catalog, its catalog cannot be read, or its catalog is damaged, so the files whose records were lost cannot all be named with FILLOST. Some files may be missing from the output.

User Action: Compare the output with the source, or with the listing of an older saveset.

UNSUPP, File: *file* - *what* not restored, errno: *n* (*reason*)

Facility: VBACKUP. **Severity:** Warning.

Explanation: A file of a type that cannot be made here was not restored: `an unknown file type`; `a special file` that could not be created (a device file restored by a user other than root, a socket); or, for `/EXTRACT`, `anything but a regular file`.

On Windows every FIFO, socket and device of a Linux saveset is said so (errno 40, Function not implemented): Windows has no such files.

User Action: Restore device files as root. Only regular files can be extracted.

VERIFYING, Saveset: *saveset* - verifying

Facility: VBACKUP. **Severity:** Informational.

Explanation: `/VERIFY`: the saveset just written is being read back and compared with the disk.

User Action: None.

VMSNOCRC, Saveset: *saveset* - written /NOCRC: its blocks carry no CRC, damage in them cannot be seen

Facility: VBACKUP. **Severity:** Informational.

Explanation: The saveset of OpenVMS BACKUP was written `/NOCRC`: its blocks have no CRC, and a damaged block is not recognized as such. Its XOR blocks, if any, are still read.

User Action: None. Check the restored files by other means where it matters.

VMSRAW, File: *file*, Organization: *org*, Record format: *rfm* - restored as it is on the VMS disk: its records are not converted

Facility: VBACKUP. **Severity:** Informational.

Explanation: A file of a saveset of OpenVMS BACKUP was restored as the image of its RMS file: an indexed or relative file, or a variable-length file without a carriage control. Its records cannot be had without RMS.

User Action: Copy the file back to OpenVMS in binary mode and set its attributes there, or `CONVERT` it to a sequential file on OpenVMS before the save.

VMSSAVESET, Saveset: *saveset*, Block size: *n*, Group size: *n* - an OpenVMS BACKUP saveset

Facility: VBACKUP. **Severity:** Informational.

Explanation: The input is a saveset written by OpenVMS BACKUP, read as Section 3.16 describes.

User Action: None.

WRITERR, File: *file*, errno: *n* - cannot be written (*reason*)

Facility: VBACKUP. **Severity:** Error.

Explanation: A write failed: a volume of the saveset (the save then ends with FATALSAVE), the catalog spool, a restored or extracted file, the standard output, the listing file, or the output device of a `/PHYSICAL` restore.

User Action: Most often the output is full (errno 28): free space, or cut the saveset into volumes, and repeat the operation.

WRONGKEY, Saveset: *saveset* - the passphrase does not open it

Facility: VBACKUP. **Severity:** Error.

Explanation: The passphrase given does not match the key check value of the encrypted saveset. Nothing was read or written. The passphrase is not kept: the next saveset of the command asks again.

User Action: Check the key file: only its first line counts, and capitals matter. A saveset whose passphrase is lost cannot be opened by anybody.

WRONGVOL, Volume: *n*, Saveset: *saveset* - belongs to another saveset, ignored

Facility: VBACKUP. **Severity:** Warning.

Explanation: A file with the name of a volume of the saveset belongs to another saveset (its saveset identifier differs), or is no volume. It is ignored; MISSVOL usually follows for that volume.

User Action: Put the right volume in its place. Do not mix volumes of savesets saved under the same name.

XATTRSKIP, File: *file* - an extended attribute not saved: it cannot be read, or is longer than 64 KB (on Windows: a stream)

Facility: VBACKUP. **Severity:** Warning.

Explanation: The file was saved, but one of its extended attributes was not: it could not be read, or its value is longer than 64 KB, the largest value an extended attribute of Linux may have. On Windows the alternate data streams of a file are its extended attributes (`user.<stream>`), and a stream longer than 64 KB is not saved.

User Action: Copy the stream into a file of its own when it matters (on Windows: `more < file:stream > file.stream`), or give `/NOXATTRS` to save without the extended attributes and without this message.

XFRSUMM, Blocks: *n*, Volumes: *n*, Bad: *n* - copied

Facility: VBACKUP. **Severity:** Informational.

Explanation: The totals of a copy of a saveset: the blocks copied, the volumes, and the bad blocks among them (each reported by BLKCOPIED). Displayed without `/LOG`, also when the copy failed, with what was copied until then. VBACKUP on another node displays it for its side of a save or a restore through *node*::*file*.

User Action: None when Bad is 0. Otherwise see BLKCOPIED.

ZCHECK, Offset: *n* - a record compressed did not decompress to the same octets: it is stored uncompressed (a fault of VBACKUP: report it)

Facility: VBACKUP. **Severity:** Warning.

Explanation: Every record VBACKUP compresses is decompressed again at once and compared with the data it was made of (Section 3.7). This one did not come back the same; it was written uncompressed, as a DATA record. The saveset is right and restores; the offset is that of the record in its file. For a group of small files (a SOLID record, see `/SOLID`) the offset is 0, and the group is written as ordinary records, uncompressed.

User Action: Report it, with the file, the `/LEVEL` and the version of VBACKUP; the data that made it helps most. Meanwhile `/LEVEL=1`, `/NOSOLID` (for a group) or `/DATA_FORMAT=UNCOMPRESSED` avoid the codec in question.

Chapter 7. Troubleshooting

This chapter starts from what is seen - a symptom or a message - and says what to look at and what to do. The messages named here are described in full in Chapter 6.

7.1 The Verify Fails on Files That Are in Use

Symptom. A save `/VERIFY` ends with the exit status 2, and the only errors are COMPARERR, often after FILCHANGED.

Cause. The files changed between being saved and being compared: a database, a log, the git tree of a program at work. The saveset holds what the files were when they were read; it is not at fault.

Action. Save a snapshot of the file system (LVM, btrfs, ZFS) instead of the live one, or run the save when nothing works on those files, or leave them out with `/EXCLUDE`. A command procedure may run the save once more when COMPARERR is the only error.

7.2 Owners and Attributes Are Not Restored

Symptom. ATTRERR for many files; the files are restored and belong to the user who restored them.

Cause. The restore was not run by root, or the file system of the output cannot hold an attribute (for example ACLs on vfat).

Action. Restore as root; or give `/OWNER=DEFAULT` to take the files as your own, and `/NOXATTRS` when the extended attributes are not wanted (Section 5.1, Restrictions).

7.3 A Saveset Is Damaged

Symptom. CRCERR, BLKFIXED, BLKLOST, FILDAMAGED or FILLOST while a saveset is read.

What to look at. BLKFIXED alone means a bad block was rebuilt from its group: everything is restored. BLKLOST means more blocks of a group were bad than the group can rebuild; the files that lay in them are named by FILDAMAGED or FILLOST, and every other file is restored. A file is never restored with wrong data without a message.

Action.

1. Copy the saveset off the failing medium first, with `dd conv=noerror,sync` if it must be.
2. Read the copy again: `vbackup copy.bck /LIST`, then restore it.
3. Try the stand-alone extractor and an extractor of last resort (Appendix C): each reads the blocks on its own.
4. For the next savesets give a larger `/PARITY` and a smaller `/GROUP_SIZE` (Section 3.3).

7.4 A Volume Is Missing or Belongs to Another Saveset

Symptom. MISSVOL, WRONGVOL, NOTRAILER.

Cause. The volumes of a saveset are looked for beside the first one, by name (Section 3.3). A volume was moved, renamed, or copied from another saveset of the same name; or the save did not complete.

Action. Put all the volumes in one directory under their names as written. The files that lie wholly in the volumes present are restored anyway. A saveset whose save did not complete has no trailer and no catalog: it is read whole, as a stream (NOCATALOG).

7.5 The Passphrase Is Asked for, Refused or Missing

Symptom. NOKEY in a batch job; WRONGKEY; KEYMATCH.

Action.

- NOKEY: there is no terminal to ask on. Give `/KEY_FILE=`*file* or set `VBACKUP_KEY_FILE`; keep the file readable by its owner alone (`chmod 600`).
- WRONGKEY: only the first line of a key file counts, and capitals matter. Check that the key file is the one the saveset was made with.
- KEYMATCH: the two answers at the terminal differed; save again.

A lost passphrase cannot be recovered: there is no other way into an encrypted saveset. Keep the key file where the savesets are not.

7.6 A Saveset on Another Node Cannot Be Reached

Symptom. REMOTE, or REMOTEERR with an exit code.

What to look at. The exit code of REMOTEERR: 2 - VBACKUP there signalled an error (its messages are shown above); 127 - `vbackup` is not on the PATH there, or the command of `VBACKUP_RSH` cannot be run here; 255 - ssh failed: the node cannot be reached, or does not let this one in by its keys.

Action. Check `ssh` *node* `vbackup /HELP` by hand: it must run without a password. The other node needs VBACKUP X01-11 or later (Section 3.10).

7.7 An Incremental Restore Is Refused

Symptom. NOTINCR, ORIGNOBASE, MISSING.

Action. `/INCREMENTAL` needs a saveset with a catalog, made by X01-02 or later; restore the full saveset first and then each incremental one in the order they were made, oldest first - not newest first as OpenVMS BACKUP does (Section 3.5). ORIGNOBASE: a saveset made before X01-02 does not say where its files came from; give an output directory instead of `/ORIGINAL`. MISSING names a file that an earlier saveset of the chain should have given: a saveset of the chain was left out.

7.8 A Device Is Refused

Symptom. PHYSMOUNTED, PHYSHELD, PHYSSMALL.

Action. A `/PHYSICAL` save takes a device that is not mounted or is mounted read-only; save a snapshot of a mounted one. A restore never writes to a mounted device or to one held by another program: unmount it first. The output device must be at least as large as the one saved (Section 3.6).

7.9 HELP Shows Only a Summary

Symptom. `vbackup /HELP` shows a short summary and no topics.

Cause. VBACKUP was built without the HELP product (Section 2.1).

Action. Install HELP and build and install VBACKUP again; or read the manual page `vbackup(1)` and this manual.

7.10 A File Manager Does Not Open a Saveset

Action. The plugins are installed into Midnight Commander and far2l only when they are found at the time of the installation; install the file manager first, then VBACKUP, or put the files of `share/vbackup/plugins` in place by hand (Appendix D). Total Commander takes the plugin from its archive (`vbackup_wcx.zip`).

Appendix A. Files

A.1 Files Installed

The paths are under the prefix, `/usr/local` by default.

File	Contents
`bin/vbackup`	The utility
`bin/vbkx`	The stand-alone extractor
`share/man/man1/vbackup.1`, `vbkx.1`	The manual pages
`share/help/vbackup.hlb`, `share/help/ru/vbackup_ru.hlb`	The help libraries, English and Russian (with HELP)
`share/doc/vbackup/`	`README.md`, `LICENSE`, `NOTICE`, `format.md`, `vmsbackup.md`, `vbackuplib.md`, `vbackuplib_ru.md`, this manual (`vbackup_users_guide.md`, `vbackup_users_guide_ru.md` and their PDF)
`share/vbackup/plugins/mc/`	`uvbk` and `mc.ext.ini.vbackup` - Midnight Commander
`share/vbackup/plugins/far/vbackup.ini`	far2l and Far Manager (MultiArc)
`share/vbackup/plugins/wcx/`	`vbackup.wcx` for Double Commander, `pluginst.inf`; `win/` - the plugin for Total Commander when it was built
`share/vbackup/plugins/magic/vbackup.magic`	The magic of a saveset for `file(1)`

Where Midnight Commander and far2l are found, the installation also puts `uvbk` into the extfs directory of Midnight Commander and its sections, between markers, into `mc.ext.ini` and into the `custom.ini` of MultiArc.

A.2 Files Used

File	Use
The journal of `/JOURNAL=`*file*	The record of the savesets and their files, for `/SINCE=BACKUP` and `/LIST /JOURNAL` (Section 3.5)
The key file of `/KEY_FILE` or `VBACKUP_KEY_FILE`	The passphrase, on its first line (Section 3.8)
`/tmp`	The spool of the catalog of a save: an unnamed temporary file, gone at the end of the save
`~/.ssh`	The keys by which VBACKUP reaches another node (Section 3.10)

The environment variables are listed in Section 5.2.

Appendix B. Saveset Format Summary

This appendix summarizes the saveset format, version 1. The authoritative definition is `doc/format.md`; a reader written from that document alone must be able to list and restore any saveset. Since X01-08 a saveset may also be read from the standard input, and since X01-11 a stream carries several volumes (see Section 3.9; `format.md`, sections 2 and 8). Version 2 (since X01-14) adds the parity of `/PARITY`; version 3 (since X01-21), the SOLID record.

B.1 Conventions

- All integers are little-endian, unsigned unless noted (u8, u16, u32, u64; i64 two’s complement). Fields are encoded byte by byte.
- TIME is 12 bytes: i64 seconds since 1970-01-01 00:00:00 UTC, u32 nanoseconds.
- Strings and paths are raw bytes without a terminating NUL; no encoding is assumed.
- CRC is CRC-32/IEEE (reflected polynomial 0xEDB88320, initial value and final XOR 0xFFFFFFFF; CRC(“123456789”) = 0xCBF43926) - not CRC-32C.
- UUID is 16 random bytes, version 4.

B.2 Volumes and Blocks

```
volume 1 : VHDR  G G G ... G
volume 2 : VHDR  G G ... G
...
volume K : VHDR  G G ... G  TRAILER

G (group) = DATA x n, then XOR        (1 <= n <= N; N = 0: no XOR blocks)
```

Volume *k* >= 2 is named *name*.*k* with at least three digits. A group never crosses a volume boundary. The block size B is 8192 to 1048576, a multiple of 512; the payload of a block is P = B - 64 bytes. Blocks are numbered from 0 through the whole saveset.

A stream (a pipe, since X01-11) carries the volumes back to back, each beginning with its VHDR; the block numbers run on as ever. A reader of a stream takes a VHDR of the next volume as the end of the volume in hand; a receiver splits the stream into volume files at each VHDR. A reader of a stream reads forward only: a TRAILER ends the groups where it comes, and the catalog at the end of the stream is not used for seeking.

B.3 Block Header (64 bytes)

Offset	Size	Field	Meaning
0	4	magic	`V B K B`
4	2	hdrlen	64
6	2	version	1; 2 with `/PARITY` of 2 or more; 3 when the saveset may hold SOLID records (a compressed save without `/NOSOLID`), with any `/PARITY`
8	4	bsize	B
12	1	type	1 DATA, 2 XOR, 3 VHDR, 4 TRAILER, 5 EDATA, 6 ETRAILER, 7 PARITY
13	1	flags	bit 0 LASTINVOL, bit 1 LASTINSET
14	2	gindex	DATA: position in the group; XOR: n; PARITY: n + 256 * row; else 0
16	16	ssuuid	UUID of the saveset
32	8	blkno	block number
40	4	volno	volume number, from 1
44	4	recoff	DATA: offset of the first record header that begins in the block, 0xFFFFFFFF if none
48	4	paylen	DATA: payload bytes used; VHDR, TRAILER: TLV body length; XOR: P
52	4	prvrecoff	recoff of the previous DATA block of the group, 0xFFFFFFFF for the first
56	4	prvpaylen	paylen of the previous DATA block of the group, 0 for the first
60	4	crc	CRC of the header (this field 0) and the whole payload area

The XOR payload is the XOR of the P-byte payload areas of the n DATA blocks of its group. One bad DATA block is rebuilt from it; its `recoff` and `paylen` come from the `prvrecoff` and `prvpaylen` of the block that follows it.

B.4 Block Types

Type	Name	Contents
1	DATA	A part of the record stream
2	XOR	Parity of the DATA blocks of its group (never encrypted)
7	PARITY	Row 1 .. m - 1 of the Reed-Solomon parity of its group, version 2 (`/PARITY`)
3	VHDR	First block of every volume: a complete SUMMARY record (encrypted saveset: a short clear SUMMARY)
4	TRAILER	Last block of the saveset: one TLV body with the totals and the place of the catalog
5	EDATA	DATA block of an encrypted saveset
6	ETRAILER	TRAILER of an encrypted saveset

B.5 Records

Record header, 8 bytes: u16 type, u16 flags (0), u32 length of the body. A record may cross block and volume boundaries; its header never does.

Type	Record	Body
1	SUMMARY	TLV items
2	FILE	TLV items (per-file tags)
3	DATA	u32 fileno, u32 0, u64 offset, data (at most 1048576 bytes)
4	FEND	TLV items: FILENO, SIZE, CRC, STATUS
5	CATALOG	entries: u32 entry length, TLV items
6	END	TLV items: NFILES, NBYTES, NERRORS
7	DATAZ	u32 fileno, u32 codec (1 = LZ4 block), u64 offset, u32 rawlen, compressed bytes
8	SOLID	u32 codec (as in DATAZ), u32 rawlen (at most 1114112), u32 count, compressed bytes: the FILE, DATA and FEND records of several small files (version 3 only; format.md 6.12)

Order in the stream:

```
SUMMARY
  ( FILE [DATA|DATAZ ...] FEND ) ...   one group per file; version 3: any
                                       run of them may be one SOLID instead
  CATALOG ...                          at most 1 MiB body each
END
```

A reader skips a record of an unknown type and a TLV item of an unknown tag by their lengths. Decompressed, a SOLID is a sequence of FILE, DATA and FEND records with their 8-byte headers, each wholly inside, the last ending at `rawlen`; a reader of version 3 delivers them as if they stood in the stream. A SOLID that does not open is a gap, like lost blocks. A SOLID cut by lost blocks, or by the end of a saveset cut short, is not all lost (VBACKUP since X01-22): the compressed bytes before the cut are decompressed as far as they are exactly right - LZ4 and Deflate up to the last whole sequence or code, LZMA up to the last symbol decoded before the input ran out, never an octet guessed - and the records wholly in that are delivered, then the gap. The writer closes a group before its records would pass 262144 bytes (1048576 in X01-21).

After a sequential pass the catalog, when there is one, is set against the FILE records met - always, not only after a loss (VBACKUP since X01-22). A file of the catalog (STATUS 0..2) whose FILE record never came is named: lost with the blocks when blocks were lost, else “the catalog has it, the stream does not” (NOTINSTREAM). A stream does it with its CATALOG records as they come (`format.md`, section 8).

B.6 Tags

A TLV item is u16 tag, u32 length, value. One tag space serves all records.

Tags	Group	Names
1-20	Per-file (FILE, CATALOG)	FILENO, PATH, FTYPE (1 REG, 2 DIR, 3 SYMLINK, 4 HARDLINK, 5 CHR, 6 BLK, 7 FIFO, 8 SOCK), MODE, UID, GID, UNAME, GNAME, SIZE, MTIME, ATIME, CTIME, BTIME, RDEV, LINK, XATTR, FSFLAGS, DEVINO, BASEIDX, NLINK
21-22	FILE (21 also CATALOG)	WINATTR (u32, FILE_ATTRIBUTE_* of Windows), NTSD (the security descriptor, self-relative) - files saved on Windows, since X01-18
23	CATALOG	SOLID (u8): 1 - the FILE record is inside the SOLID record at LOCVOL, LOCBLK, LOCOFF
32-36	FEND, CATALOG	CRC, STATUS (0 OK, 1 CHANGED, 2 READERR, 3 PRESENT - catalog only), LOCVOL, LOCBLK, LOCOFF
64-77	SUMMARY	PRODUCT, HOST, USER, CMDLINE, CREATED, BASE, BLOCKSIZE, GROUPSIZE, VOLSIZE, COMMENT, SYSTEM, KIND (0 FULL, 1 INCREMENTAL), FILTER, COMPRESS
78-80	SUMMARY, /PHYSICAL	PHYSICAL, DEVSIZE, SECTORSIZE
81-87	SUMMARY, /IMAGE	IMAGE, FSTYPE, FSLABEL, FSUUID, FSUSED, ROOTATTR, MOUNTOPTS
88-92	VHDR and SUMMARY, encryption	CIPHER (1), KDF (1), KDFITER, SALT, KEYCHECK
96-104	END, TRAILER	NFILES, NBYTES, NERRORS, NBLOCKS, CATVOL, CATBLK, CATOFF, NVOLS, NENTRIES
128-130	Journal	SSUUID, SPEC, RECORDED

B.7 Encryption (format.md, 6.10)

- Primitives: ChaCha20 (RFC 8439) and SHA-256 (FIPS 180-4), with HMAC-SHA256 and PBKDF2-HMAC-SHA256 built from it.
- Keys:

```
MK    = PBKDF2-HMAC-SHA256(passphrase, SALT, KDFITER, 32 bytes)
KENC  = HMAC-SHA256(MK, "VBACKUP ENC")
KMAC  = HMAC-SHA256(MK, "VBACKUP MAC")
CHECK = HMAC-SHA256(MK, "VBACKUP CHECK")     stored as KEYCHECK
```

- Payload area of EDATA and ETRAILER: the ciphertext (`paylen` bytes), zeros, and a 32-byte TAG at P - 32; `paylen` is at most P - 32.
- Encryption: ciphertext = plaintext XOR ChaCha20(KENC, nonce = u32 0 || u64 blkno, counter from 0).
- TAG = HMAC-SHA256(KMAC, ssuuid || bsize || blkno || volno || type || gindex || recoff || paylen || ciphertext). A block with a good CRC and a wrong TAG is a bad block. The TAG is checked before decryption.
- The VHDR carries in the clear only PRODUCT, BLOCKSIZE, GROUPSIZE, VOLSIZE, CIPHER, KDF, KDFITER, SALT and KEYCHECK; the complete SUMMARY is the first record of the encrypted stream.
- CRCs and XOR blocks are computed over the ciphertext: checking and repair need no passphrase.

B.8 The Journal

A file of its own: a 16-byte header (`V B K J`, u16 version 1, u16 reserved, u64 record count), records of the saveset record format - SSET (type 16) for a saveset, FSTATE (type 17) for a file - and a final u32 CRC of everything before it. The key of an FSTATE is the absolute name of the file. The journal is rewritten whole through `journal.tmp` under an exclusive lock on `journal.lock`.

B.9 OpenVMS BACKUP Savesets

Read, not written (Section 3.16; `doc/vmsbackup.md`): blocks of one size with a 256-byte header (BBH: size 256, structure level 0x0101, applic 1 for data and 2 for XOR, the block number from 1, CRC-32/IEEE of the block with its CRC and checksum fields as zeroes, flag 1 `/NOCRC`), records with a 16-byte header (BRH: size, type 1 SUMMARY, 3 FILE, 4 VBN, the VBN address), SUMMARY and FILE attributes as counted TLVs, the data of a file as its VBNs of 512 octets.

Appendix C. The Stand-Alone Extractor vbkx

C.1 Description

vbkx reads savesets made by VBACKUP on a machine where VBACKUP is not installed: a rescue system, another distribution, a file manager. It is built from the reading core of VBACKUP only - no command language, no help library, no message facility - and is linked statically where the system allows it: copy the one file and run it.

vbkx of X01-21 reads savesets of format version 3 - the SOLID records of a compressed save (Section 3.7, `/SOLID`) - as well as versions 1 and 2, in all its commands: `l`, `x`, `p`, `t`. A file named to `x` or `p` is reached through the catalog by its SOLID, which is decompressed, the files before it passed. vbkx before X01-21 refuses such a saveset (`is not a saveset`). vbkx of X01-22 restores the files of a SOLID before the place where lost blocks cut it, as VBACKUP does (Section 3.7), and with `x` and `t` names every file of the catalog the stream did not give.

C.2 Format

```
vbkx l saveset [-m] [-k keyfile] [-n]
vbkx x saveset [-C dir] [-f] [-j] [-k keyfile] [-n] [name ...]
vbkx p saveset name [-k keyfile] [-n]
vbkx t saveset [-k keyfile] [-n]
```

The command letter comes first, then the saveset (the first volume, or `-` for the standard input), then the options and names in any order.

C.3 Commands

Command	Action
`l`	Lists the files, one line each, from the catalog (or from the stream when there is none).
`x`	Extracts all files in one pass, or only the names given, each reached through the catalog. A directory name takes everything below it.
`p`	Writes one file to the standard output.
`t`	Reads the whole saveset and checks every checksum; on success displays *saveset*`: all files read, all checksums match`. A file of the catalog the stream did not give is named, and the check fails.

The listing has the form, the time being local:

```
2026-10-05 14:19:25         4096 d0755 rrl
2026-10-05 14:19:25            6 -0644 rrl/a.txt
2026-10-05 14:19:25            5 l0777 rrl/link -> a.txt
```

The type letters are those of `ls: - d l h` (hard link, followed by `link to` *name*) `c b p s`, then the mode in octal.

C.4 Options

Option	Applies to	Meaning
`-C dir`	x	Extracts into *dir*, made if missing; default the current directory.
`-f`	x	Overwrites files that exist; without it they are kept and said.
`-j`	x	Extracts the files by their last name component, without directories (MultiArc “extract without path”).
`-m`	l	Lists without `-> target`, for programs.
`-k keyfile`	all	The passphrase of an encrypted saveset: the first line of *keyfile*, which only its owner may read (mode 600).
`-n`	all	Never asks for a passphrase (for programs); `VBACKUP_NOPROMPT=1` does the same.

Without `-k`, the key file named by `VBACKUP_KEY_FILE` is used; without both, the passphrase is asked at the terminal (on Windows, the console).

C.5 What Is Restored

vbkx restores the data and holes, the mode, the modification and access times, directories, symbolic and hard links and FIFOs; as root also the owner (by number) and device files. It does not restore ACLs, extended attributes or chattr flags; restore with VBACKUP when they are needed.

The names of a saveset are not trusted: a name with `..`, a leading `/`, or a way through a symbolic link is refused, and every directory on the way is opened with O_NOFOLLOW. Damage is repaired as far as the XOR blocks allow; every file cut short is named `is incomplete`, every file of the catalog that could not be reached `not extracted` - after lost blocks cut a SOLID, the files of it from the damage on; those before are extracted. The catalog is always set against the stream (since X01-22): a file of the catalog whose records never came, though no block was lost, is named `not extracted: the catalog has it, the record stream does not` (as NOTINSTREAM of VBACKUP). From a pipe (`-`) the saveset is listed and extracted as it is read; names are refused, because its catalog comes last; its catalog records are set against the stream as they come.

C.6 Diagnostics

vbkx writes its diagnostics to the standard error, prefixed `vbkx:`, in the form of the VBACKUP messages - `Label: value - words` - but without the date, process number and message identifier:

```
vbkx: Block: 3, Volume: 1 - is bad and cannot be rebuilt
vbkx: File: rrl/big.bin - checksum mismatch: the data differ from what was saved
vbkx: File: rrl/big.bin - is incomplete: its data was lost in bad blocks
vbkx: File: rrl/c.txt - not extracted: its records were lost in bad blocks
vbkx: File: rrl/d.txt - not extracted: the catalog has it, the record stream does not
vbkx: Saveset: e.bck - the passphrase does not open it
```

C.7 Completion Codes

Code	Meaning
0	Done.
1	Something was damaged or not done (a file incomplete, not extracted, existing, an attribute not set).
2	The command or the saveset cannot be used (wrong command, not a saveset, cannot be opened, wrong or missing passphrase, output directory cannot be made).

C.8 vbkx.exe on Windows

vbkx.exe takes the same commands. The names, UTF-8 in the saveset, become UTF-16 under `\\?\`, so long paths work. It restores the data, sizes, times, the read-only attribute (no write bit for the owner), directories and hard links; symbolic links where Windows allows them (developer mode, or the right to make them), otherwise said. FIFOs, device files, owners and modes beyond read-only are not made. A name Windows cannot hold - a component with `<>:"\|?*` or a control character, a device name (CON, PRN, AUX, NUL, COM1-9, LPT1-9, with an extension too), a name ending in a period or a space - is not extracted, and said; a reparse point on the way is refused. The data and the listing are written byte for byte as on Linux.

vbkx.exe needs no StarLet and no CMake. With MinGW-w64, from the top of the source tree:

```
$ x86_64-w64-mingw32-gcc -O2 -Ilib -o vbkx.exe tools/vbkx.c \
    lib/vbkfmt.c lib/vbkrd.c lib/vbklz4.c lib/vbkdfl.c lib/vbklzm.c lib/vbkcrp.c lib/vbkvms.c lib/vbkrs.c
static -lshell32
$ make -f tools/Makefile.win
```

C.9 The Extractors of Last Resort: vbkx-go, vbkx-rs, vbkx-pl

Three further extractors are kept for the day everything else is dead:

Program	Source	Build
vbkx-go	`tools/go/main.go`	`go build -o vbkx-go main.go` (Go 1.19 or later)
vbkx-rs	`tools/rust/src/main.rs`	`rustc -O -C strip=symbols -o vbkx-rs src/main.rs` (Rust 1.63 or later), or `cargo build --release`
vbkx-pl	`tools/perl/vbkx.pl`	Nothing to build: `perl vbkx.pl ...` (Perl 5.10 or later, the modules of `perl-base` only)

`make` in `tools/go` and `tools/rust` does the same; the CMake build makes vbkx-go and vbkx-rs when Go or Rust is found. Each is one source file using the standard library only, plain on purpose so that it can be read and corrected against `format.md`; the head of each source is its manual.

Format

```
vbkx-go l saveset [-k file]
vbkx-go x saveset [-C dir] [-k file]
vbkx-go t saveset [-k file]
vbkx-go selftest
```

(the same for `vbkx-rs` and `perl vbkx.pl`)

Command	Action
`l`	Lists the files from the FILE records of the stream (no catalog needed), times in UTC.
`x`	Extracts all files into *dir* (default the current directory). An existing file is never overwritten.
`t`	Reads the whole saveset and checks the checksums.
`selftest`	Checks SHA-256, HMAC-SHA256, PBKDF2 and ChaCha20 against the vectors of their standards.

The listing is the same in all three, byte for byte:

```
2026-10-03 20:40:12         1234 -0644 tree/a.txt
2026-10-03 20:40:12            5 l0777 tree/link -> a.txt
2026-10-03 20:40:12            6 h0644 tree/hard link to tree/a.txt
```

They restore data (holes stay holes), modes with the setuid, setgid and sticky bits, modification and access times, directories, symbolic links, hard links and FIFOs; not owners, ACLs, extended attributes, chattr flags, device files or sockets. They read DATAZ records with their own checked LZ4 decoder, and encrypted savesets with `-k keyfile`, else `VBACKUP_KEY_FILE`, else a passphrase asked at the terminal without echo; every block is checked by its tag before it is decrypted, and a forged block is rebuilt from its group like a bad one. They repair one bad block per group, resume after a loss at the next good block, skip a missing or cut volume, find the block size by trying when the first block of volume

1 is lost, and name every damaged (`is incomplete`) or missing (`was not extracted`) file. A `/PHYSICAL` saveset comes out as the image file of the device, an `/IMAGE` saveset as the tree of its files. The three of X01-21 read savesets of version 3: each opens a SOLID record with its own decoder and takes its records as if they stood in the stream; a SOLID that does not open is a gap, and its files are named as missing. They do not salvage a SOLID cut by lost blocks, as VBACKUP and vbkx do since X01-22: they lose the whole SOLID, and still name every file of it.

vbkx-pl uses Digest::SHA when it is installed, only for speed; without it its own SHA-256 is some hundred times slower (opening an encrypted saveset then takes minutes). `VBKXPL_PURE=1` makes it use its own code even when Digest::SHA is there. It reads a few megabytes a second.

Completion codes of all three: 0 - all done; 1 - something was damaged or not done; 2 - the command or the saveset cannot be used.

Appendix D. File Manager Plugins

D.1 General

A saveset opens like a folder in the file managers below: list, view, copy out, test. All plugins are read only: nothing is ever written into a saveset. The listing comes from the catalog, so a saveset of hundreds of gigabytes opens at once, and a file is copied out through its place in the catalog - for a small file of a compressed save, through its SOLID record (Section 3.7). The plugins of X01-21 read savesets of format version 3; older ones do not open them. The plugins share the reader of VBACKUP: since X01-22 they too restore the files of a SOLID before the place where lost blocks cut it.

A file manager cannot ask for a passphrase. To open an encrypted saveset, put the passphrase into a key file (`chmod 600`) and name the file in the environment variable `VBACKUP_KEY_FILE` before the file manager is started (on Windows, in the system settings of environment variables). Without it the saveset does not open; nothing waits for an answer on the screen. The plugins set `VBACKUP_NOPROMPT=1` or give vbkx `-n`.

D.2 Midnight Commander

Item	Value
Files	`plugins/mc/uvbk` (extfs script), `plugins/mc/mc.ext.ini.vbackup` (section of `mc.ext.ini`)
Works over	`vbackup` (the variable `VBACKUP` may name another image)
Installation	`cmake --install` puts `uvbk` into the extfs directory of MC and the section into `mc.ext.ini` when MC is present, and removes them on uninstall. By hand: copy `share/vbackup/plugins/mc/uvbk` into `~/.local/share/mc/extfs.d` (or `/usr/lib/mc/extfs.d`) and the section into `mc.ext.ini`, before `[Default]`.
Use	Enter on a `.bck` file; F3 shows the full listing.

The section:

```
[vbackup]
Shell=.bck
ShellIgnoreCase=true
Open=%cd %p/uvbk://
View=%view{ascii} vbackup %f /SAVE_SET /LIST /FULL
```

`uvbk list` gives the catalog as `ls -l` lines (`vbackup ARCHIVE /LIST /FORMAT=LS`); `uvbk copyout` takes one file (`vbackup ARCHIVE TARGET /EXTRACT=NAME`). Copying in, deleting and making directories are refused. A saveset of several volumes opens from a local disk only: inside another VFS (an archive, FTP) MC gives the script a temporary copy of volume 1, and the other volumes are not beside it.

D.3 far2l and Far Manager 3

Item	Value
File	`plugins/far/vbackup.ini`, a format description for the MultiArc plugin
Works over	vbkx (`vbkx.exe` on Windows, found on PATH)
Installation	far2l: appended to `~/.config/far2l/plugins/multiarc/custom.ini` (the installation does it). Far3: append it to `Plugins\MultiArc\Formats\custom.ini`.
Use	Enter or Ctrl/PgDn on a `.bck` file; F5 copies files out.

The commands MultiArc runs:

```
List=vbkx l %%AQ -m -n
Extract=vbkx x %%AQ -f -n %%FSQ
ExtractWithoutPath=vbkx x %%AQ -f -j -n %%FSQ
Test=vbkx t %%AQ -n
```

A saveset is recognized by its first four bytes, `VBKB` (`ID=56 42 4B 42`), and by the extension `bck`.

D.4 Total Commander and Double Commander

Item	Value
Source	`plugins/wcx/vbkwcx.c`, a WCX packer plugin over the reading core
Files	`vbackup.wcx64` (Total Commander 64-bit), `vbackup.wcx` (Total Commander 32-bit), `vbackup.wcx` - a shared object - for Double Commander on Linux
Build	`make -f plugins/wcx/Makefile linux win64 win32` (into `out/linux`, `out/win64`, `out/win32`); the CMake build makes the Linux one always (installed under `share/vbackup/plugins/wcx`) and the Windows ones when MinGW-w64 is found
Installation, TC	Open the plugin archive (`make -f plugins/wcx/Makefile zip`, with `pluginst.inf`) in TC and confirm; or Configuration -> Options -> Packer -> Configure packer extension WCXs, extension `bck`.
Installation, DC	Options -> Plugins -> Packer plugins (WCX) -> Add, extension `bck`.

The plugin lists from the catalog (without one, or with a damaged one, it reads the saveset through), copies a file out through its place in the catalog and checks its FILENO and CRC, reads DATA and DATAZ records and holes, repairs what the XOR blocks allow and reports a file that does not come out whole (E_BAD_DATA). It asks for a volume that is not beside volume 1 through the ChangeVol callback. A name that would lead out of the target (`..`, a leading `/`, a backslash; on Windows also `c:`, `con`, `x?`) is not listed. Nothing is written outside the target.

Limitations: an encrypted saveset is opened only with `VBACKUP_KEY_FILE` (without it E_EOPEN, with a wrong passphrase E_BAD_ARCHIVE); owners, ACLs, extended attributes and chattr flags are not restored; device files, FIFOs and sockets are listed but not made; symbolic links are made on Linux and skipped on Windows; the times of the listing are DOS times (local, two seconds), while an extracted file gets its times in full.